



Norme di Progetto

The Bitless (Gruppo 18):

Lorenzo Battistella (2103116), Edoardo De Piccoli (2101055), Davide Facco (2087852), Anna Marini (2110986), Andrea Menegaldo (2116426), Samuele Vendramin (2111934), Simone Zecchinato (2113189)

Informazioni documento			
Versione	Data	Stato	Destinatari
0.5.0	2026-04-29	In stesura	The Bitless, prof. Tullio Vardanega, prof. Riccardo Cardin

Contatti: thebitless.swe@gmail.com

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.18.1	2026-05-13	A. Marini	S. Vendramin	-	Corretto formato riferimenti
0.18.0	2026-05-11	D. Facco	S. Vendramin	-	Sistemazione Attività di verifica (sezione 3.3.2)
0.17.1	2026-05-08	E. De Piccoli	S. Vendramin	-	Correzione errori di battitura
0.17.0	2026-05-07	L. Battistella	S. Vendramin	-	Stesura sezione Metriche per la Qualità del prodotto
0.16.0	2026-05-04	S. Zecchinato	S. Vendramin	-	Stesura sezione Metriche di Qualità del processo
0.15.1	2026-05-03	A. Menegaldo	S. Vendramin	-	Correzioni grammaticali e adeguamento formattazione tabelle
0.15.0	2026-05-02	S. Vendramin	D. Facco	-	Fine sezione Metriche per la qualità
0.14.0	2026-04-30	A. Marini	S. Vendramin	-	Stesura sezione Formazione e inizio Metriche per la qualità
0.13.0	2026-04-28	A. Menegaldo	S. Vendramin	-	Stesura sezione 3.4 Gestione della configurazione
0.12.0	2026-04-26	L. Battistella	E. De Piccoli	-	Stesura sezione Coordinamento e Miglioramento
0.11.1	2026-04-24	S. Zecchinato	E. De Piccoli	-	Aggiornamento sezione Struttura verbali
0.11.0	2026-04-22	S. Zecchinato	E. De Piccoli	-	Stesura Gestione dei Processi
0.10.0	2026-04-18	S. Vendramin	E. De Piccoli	-	Stesura sezione 3.5 Risoluzione dei problemi
0.9.0	2026-04-16	E. De Piccoli	A. Menegaldo	-	Stesura sezione 3.3 Validazione
0.8.0	2026-04-15	A. Marini	A. Menegaldo	-	Stesura sezione 3.2 Verifica
0.7.0	2026-04-13	S. Zecchinato	A. Menegaldo	-	Stesura sezione 3.1 Documentazione

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.6.1	2026-04-10	A. Marini	A. Menegaldo	-	Correzione errori ortografici sezione Fornitura
0.6.0	2026-04-08	D. Facco	A. Menegaldo	-	Stesura processo di Sviluppo (Processi primari)
0.5.0	2026-04-04	S. Vendramin	A. Menegaldo	-	Stesura processo di Fornitura (Processi primari)
0.4.0	2026-04-01	L. Battistella	A. Menegaldo	-	Struttura del documento e stesura Introduzione

Indice

1	Introduzione	10
1.1	Scopo del documento	10
1.2	Scopo del prodotto	10
1.3	Glossario	11
1.4	Riferimenti	11
1.4.1	Riferimenti normativi	11
1.4.2	Riferimenti informativi	11
2	Processi Primari	11
2.1	Fornitura	12
2.1.1	Processo di Fornitura	12
2.1.2	Attività	12
2.1.3	Documentazione fornita	13
2.1.4	Procedure Operative	14
2.1.5	Strumenti	14
2.2	Sviluppo	15
2.2.1	Processo di sviluppo	15
2.2.2	Attività previste	15
2.2.3	Procedure operative	16
2.2.3.1	Analisi dei Requisiti	16
2.2.3.1.1	Descrizione	16
2.2.3.1.2	Obiettivi	17
2.2.3.1.3	Documentazione	17
2.2.3.2	Casi d'uso	18
2.2.3.3	Diagrammi dei casi d'uso	19
2.2.3.4	Requisiti	20
2.2.3.5	Progettazione	21
2.2.3.5.1	Descrizione	21
2.2.3.5.2	Obiettivi	21
2.2.3.5.3	Documentazione	22
2.2.3.6	Qualità dell'architettura	22
2.2.3.7	Diagrammi UML	23
2.2.3.8	Design Pattern	25
2.2.3.9	Codifica	25

2.2.3.9.1	Descrizione	25
2.2.3.9.2	Obiettivi	25
2.2.3.10	Norme di codifica	25
2.2.3.11	Configurazione Ambiente di esecuzione	27
2.2.4	Strumenti a supporto	27
3	Processi di supporto	27
3.1	Documentazione	28
3.2	Documentazione	28
3.2.1	Processo	28
3.2.1.1	Documentation as Code	29
3.2.2	Ciclo di vita dei documenti	30
3.2.2.1	Stato di necessità	31
3.2.2.2	Pianificazione	31
3.2.2.3	Stato di redazione	32
3.2.2.4	Stato di verifica	33
3.2.2.5	Stato di approvazione	34
3.2.2.6	Stato di manutenzione	34
3.2.3	Attività previste	35
3.2.3.1	Produzione della documentazione	35
3.2.3.2	Revisione	35
3.2.3.3	Approvazione	36
3.2.3.4	Archiviazione e pubblicazione	36
3.2.3.5	Compiti del Responsabile	36
3.2.3.6	Compiti dell'Amministratore	37
3.2.3.7	Redazione dei verbali	37
3.2.3.8	Redazione dell'Analisi dei Requisiti	38
3.2.4	Procedure operative	38
3.2.4.1	Struttura generale dei documenti	38
3.2.4.1.1	Frontespizio	38
3.2.4.1.2	Registro delle modifiche	39
3.2.4.1.3	Indice	39
3.2.4.2	Struttura dei Verbali	39
3.2.4.2.1	Informazioni sulla riunione	39
3.2.4.2.2	Presenze	40

3.2.4.2.3	Ordine del giorno	40
3.2.4.2.4	Discussione e sintesi	40
3.2.4.2.5	Attività da svolgere	40
3.2.4.3	Versionamento	40
3.2.4.3.1	Branch	40
3.2.4.3.2	Stato dei documenti	41
3.2.4.3.3	Nomenclatura	41
3.2.4.3.4	Commit e Pull Request	41
3.2.4.4	Processo di verifica tramite Pull Request	42
3.2.4.5	Processo di approvazione tramite Pull Request	42
3.2.4.6	Acronimi	43
3.2.5	Strumenti di supporto	44
3.2.5.1	$\text{\LaTeX}_G$	44
3.2.5.2	GitHub $_G$	44
3.2.5.3	Visual Studio Code	45
3.2.5.4	Notion	45
3.3	Verifica	45
3.3.1	Processo di verifica	45
3.3.2	Attività di verifica	46
3.3.2.1	Verifica dei documenti	46
3.3.2.2	Analisi statica	46
3.3.2.3	Walkthrough	47
3.3.2.4	Inspection	47
3.3.2.5	Analisi dinamica e testing	47
3.3.2.6	Test di unità	48
3.3.2.7	Test di integrazione	48
3.3.2.8	Test di sistema	48
3.3.2.9	Test di regressione	48
3.3.2.10	Test di accettazione	48
3.3.2.11	Sequenza delle fasi di test	49
3.3.2.12	Codici identificativi dei test	49
3.3.2.13	Stato dei test	49
3.3.2.14	Integrazione continua	50
3.3.3	Procedure operative	50
3.3.3.1	Procedura di verifica documentale	50

3.3.3.2	Procedura di walkthrough	50
3.3.4	Strumenti di supporto	51
3.3.4.1	Test frontend	51
3.3.4.2	Test backend	51
3.4	Validazione	52
3.4.1	Processo di validazione	52
3.4.2	Attività di validazione	52
3.4.3	Procedure operative	53
3.4.4	Strumenti di supporto	54
3.5	Gestione della configurazione	54
3.5.1	Processo di gestione della configurazione	54
3.5.2	Attività di gestione della configurazione	55
3.5.2.1	Identificazione dei Configuration Item	55
3.5.2.2	Versionamento	55
3.5.2.3	Gestione dei repository	56
3.5.3	Procedure operative	56
3.5.3.1	Workflow di sviluppo: Gitflow	56
3.5.3.2	Gestione delle Pull Request	57
3.5.3.3	Controllo di configurazione e Change Request	57
3.5.4	Strumenti di supporto	58
3.5.4.1	Git	59
3.5.4.2	GitHub	59
3.6	Risoluzione dei problemi	59
3.6.1	Processo di risoluzione dei problemi	59
3.6.2	Attività di risoluzione dei problemi	60
3.6.2.1	Gestione dei rischi	60
3.6.2.2	Identificazione e tracciamento dei problemi	61
3.6.3	Procedure operative	61
3.6.3.1	Codifica dei rischi	61
3.6.3.2	Procedura di segnalazione e gestione dei problemi	62
3.6.4	Strumenti di supporto	62

4	Processi organizzativi	63
4.1	Gestione dei Processi	63
4.1.1	Processo di gestione dei processi	63
4.1.2	Attività di gestione	64
4.1.2.1	Pianificazione	64
4.1.2.2	Assegnazione dei ruoli e responsabilità	64
4.1.2.3	Ticketing e tracciamento delle attività	65
4.1.3	Procedure operative	66
4.1.3.1	Procedura di pianificazione	66
4.1.3.2	Procedura di gestione del ciclo di vita delle issue	66
4.1.4	Strumenti di supporto	67
4.2	Coordinamento	67
4.2.1	Processo di coordinamento	67
4.2.2	Attività di coordinamento	68
4.2.2.1	Comunicazioni sincrone	68
4.2.2.2	Comunicazioni asincrone	68
4.2.2.3	Riunioni interne	68
4.2.2.4	Riunioni esterne	69
4.2.3	Procedure operative	69
4.2.3.1	Procedura per le riunioni interne	69
4.2.3.2	Procedura per le riunioni esterne	70
4.2.4	Strumenti di supporto	70
4.3	Miglioramento	71
4.3.1	Processo di miglioramento	71
4.3.2	Attività di miglioramento	71
4.3.2.1	Analisi	72
4.3.2.2	Miglioramento	72
4.3.3	Procedure operative	72
4.3.3.1	Procedura di retrospettiva e miglioramento	72
4.3.4	Strumenti di supporto	73
4.4	Formazione	73
4.4.1	Processo di formazione	73
4.4.2	Attività di formazione	74
4.4.2.1	Formazione individuale	74
4.4.2.2	Formazione di gruppo	74

4.4.3	Procedure operative	75
4.4.3.1	Procedura per il passaggio di consegne (rotazione dei ruoli) . . .	75
4.4.3.2	Procedura per le sessioni formative con la proponente	75
4.4.4	Strumenti di supporto	76
5	Metriche e standard per la Qualità	76
5.1	Funzionalità	76
5.2	Affidabilità	77
5.3	Efficienza	77
5.4	Usabilità	77
5.5	Manutenibilità	77
5.6	Portabilità	78
5.7	Nomenclatura delle Metriche	78
5.8	Suddivisione secondo ISO/IEC 12207	79
5.8.0.1	Riferimenti	79
6	Metriche di Qualità del Processo	79
6.1	Processi primari	79
6.1.1	Fornitura	79
6.1.1.1	Earned Value (EV)	79
6.1.1.2	Planned Value (PV)	80
6.1.1.3	Actual Cost (AC)	80
6.1.1.4	Cost Performance Index (CPI)	81
6.1.1.5	Schedule Performance Index (SPI)	81
6.1.1.6	Estimate At Completion (EAC)	82
6.1.1.7	Estimate To Complete (ETC)	82
6.1.1.8	Time Estimate At Completion (TEAC)	83
6.1.2	Sviluppo	83
6.1.2.1	Requisiti Stability Index (RSI)	83
6.2	Processi di supporto	84
6.2.1	Documentazione	84
6.2.1.1	Indice di Gulpease (IG)	84
6.2.1.2	Indice di Frammentazione (IF)	85
6.2.2	Verifica	86
6.2.2.1	Test Success Rate (TSR)	86
6.2.3	Gestione della Qualità	87

6.2.3.1	Percentuale Metriche Soddisfatte (PMS)	87
6.3	Processi organizzativi	87
6.3.1	Gestione dei processi	87
6.3.1.1	Percentuale Rischi Inattesi (PRI)	87
6.3.1.2	Labor Efficiency (LE)	88
7	Metriche di Qualità del Prodotto	89
7.1	Funzionalità	89
7.1.0.1	Percentuale Requisiti Obbligatori Soddisfatti (PROS)	89
7.1.0.2	Percentuale Requisiti Desiderabili Soddisfatti (PRDS)	89
7.2	Affidabilità	90
7.2.0.1	Line Coverage (LC)	90
7.3	Usabilità	90
7.3.0.1	Profondità Massima di Navigazione (PMN)	90
7.4	Efficienza	91
7.4.0.1	Tempo Medio di Risposta (TMR)	91
7.5	Manutenibilità	92
7.5.0.1	Complessità Ciclomatica (CC)	92
7.6	Portabilità	92
7.6.0.1	Compatibilità Cross-Browser (CCB)	92

1 Introduzione

1.1 Scopo del documento

Questo documento delinea le modalità operative, denominate Way of Working_G, adottate dal team **The Bitless** durante lo svolgimento del progetto didattico. Il suo scopo è stabilire un protocollo comune di lavoro: una serie di norme che ogni componente del gruppo deve rispettare. Questo garantisce che lo sviluppo proceda in modo fluido, ordinato e con uno standard qualitativo elevato.

Il documento recepisce le direttive del Regolamento del Progetto Didattico e si ispira allo standard internazionale **ISO/IEC 12207:1995**. Tale norma individua tre principali tipologie di processi:

- **Processi Primari:** tutto ciò che serve a creare il prodotto, dalla raccolta dei requisiti fino alla manutenzione finale;
- **Processi di Supporto:** attività trasversali che garantiscono la solidità del lavoro, come la gestione della configurazione e le attività di test;
- **Processi Organizzativi:** compiti dedicati alla gestione delle risorse, alla formazione interna e al costante miglioramento dell'infrastruttura di squadra.

In linea con lo standard, è stato applicato un processo di **personalizzazione (tailoring_G)**, selezionando esclusivamente gli elementi necessari al nostro contesto specifico. Il documento seguirà un approccio **incrementale**, venendo aggiornato e perfezionato man mano che il team acquisisce nuove competenze e feedback.

1.2 Scopo del prodotto

Il progetto proposto dal capitolato **C6** di **Zucchetti S.p.A.**, consiste nella realizzazione di **Second Brain**: una piattaforma digitale evoluta basata sullo standard **Markdown_G**.

L'elemento distintivo del sistema risiede nell'integrazione nativa di modelli di linguaggio estesi (**LLM_G**), finalizzati ad abilitare il paradigma del **"Distant Writing"_G**. In questo scenario, l'utente assume il ruolo strategico di **progettista del testo** e coordinatore del flusso narrativo, delegando all'intelligenza artificiale l'esecuzione iterativa e l'espansione dei nuclei informativi.

L'apporto degli LLM si declina in due aree funzionali:

- **Collaborazione Iterativa e Generativa:** Il sistema supporta l'utente nelle operazioni di riscrittura adattiva, sintesi semantica, traduzione multilingue ed estrazione automatica di entità e concetti rilevanti, ottimizzando i tempi di produzione dei contenuti.
- **Analisi Critica:** Attraverso l'applicazione della tecnica dei **"Sei Cappelli per Pensare"**, l'assistente virtuale agisce come un revisore analitico capace di esaminare i testi da diverse angolazioni (rischi, benefici, logica e creatività), offrendo un supporto decisionale strutturato che eleva la qualità del prodotto finale.

Il termine ultimo per la consegna del prodotto è fissato per il **30 agosto 2026**, con un budget operativo previsto di **12.860,00 €**.

1.3 Glossario

La creazione e lo sviluppo di un sistema software richiedono una complessa operazione di progettazione e analisi del dominio, attività che deve necessariamente precedere la stesura del codice. Il gruppo si impegna a raccogliere e organizzare tali informazioni in modo che siano facilmente accessibili, al fine di favorire la massima asincronia ed efficienza nelle attività di progetto.

La principale fonte di ambiguità nello svolgimento delle attività è rappresentata dall'incomprensione dei termini tecnici o specialistici adottati. A tale scopo, la nomenclatura di riferimento viene raccolta nel *glossario*, un documento incrementale volto a definire ogni termine rilevante per il dominio del progetto.

Il gruppo si impegna a contrassegnare ogni occorrenza di un termine presente nel glossario mediante l'apposizione di una lettera **G** a pedice, come esemplificato di seguito:

parola_G

Al fine di garantire una corretta comprensione del dominio, è fondamentale che ogni membro del gruppo consulti periodicamente il glossario, assicurando così il costante allineamento sulle definizioni e sui concetti chiave.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- **Capitolato C6 - Second Brain:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
Ultimo accesso: 8 maggio 2026.

1.4.2 Riferimenti informativi

- **Regolamento del Progetto Didattico:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
Ultimo accesso: 8 maggio 2026.
- **Standard ISO/IEC 12207:1995:**
https://www.math.unipd.it/~tullio/IS-1/2009/Approfondimenti/ISO_12207-1995.pdf
Ultimo accesso: 8 maggio 2026.
Note: Fornisce la struttura dei processi del ciclo di vita del software adottata dal gruppo.
- **Glossario v0.5.0:**
https://thebitless.github.io/RTB/doc_interna/glossario/glossario.pdf
Ultimo accesso: 8 maggio 2026.

2 Processi Primari

In conformità allo standard **ISO/IEC 12207:1995**, i processi primari costituiscono le attività essenziali che coprono l'intero ciclo di vita del software. Essi definiscono le responsabilità operative e gestionali delle figure coinvolte nella realizzazione del progetto.

Il gruppo **The Bitless** ha selezionato i seguenti processi come cardini del proprio *Way of Working_G*:

- **Fornitura** (Supply Process);
- **Sviluppo** (Development Process).

2.1 Fornitura

2.1.1 Processo di Fornitura

Il processo di **fornitura** descrive il modo in cui il gruppo gestisce il lavoro per il cliente, dalla pianificazione iniziale fino alla consegna del software. Esso comprende tutte le fasi di comunicazione con i committenti, la firma degli accordi, l'organizzazione dei compiti e il controllo costante che tutto proceda secondo i tempi stabiliti.

L'obiettivo principale è assicurare che ogni fase del lavoro sia chiara e documentata. In questo modo, il gruppo può garantire che il prodotto finale rispetti esattamente le richieste e gli standard di qualità promessi a **Zucchetti S.p.A.** durante la fase di analisi.

2.1.2 Attività

In linea con lo standard, il processo di fornitura si articola in sette attività principali:

1. **Initiation:** rappresenta il punto di partenza in cui il gruppo compie un'analisi approfondita delle richieste avanzate dal proponente (**RFP_G**). Gli studenti valutano i requisiti per determinare la fattibilità_G del progetto e verificare che il team possieda le competenze tecniche necessarie per procedere.
2. **Preparation of response:** fase dedicata alla redazione della proposta progettuale. In questo stadio viene definito il **Tailoring_G** dello standard, selezionando esclusivamente i processi e i compiti realmente funzionali agli obiettivi del gruppo, mantenendo il rigore metodologico richiesto.
3. **Contract:** accordo formale tra il gruppo, il docente e il tutor aziendale. Vengono definiti con precisione i **deliverable_G**, le scadenze temporali e i criteri di accettazione per pervenire alla formalizzazione del contratto sui requisiti e alla stipula definitiva.
4. **Planning:** fase in cui il team organizza il lavoro internamente definendo il **framework_G** di gestione. Si sceglie il modello di sviluppo, si distribuiscono i compiti, si allocano le risorse e si effettua un'analisi preventiva dei rischi tecnici e temporali.
5. **Execution and control:** attuazione operativa dei piani definiti e produzione vera e propria di codice e documentazione. Il gruppo monitora costantemente lo stato di avanzamento del lavoro, il rispetto del budget_G e delle tempistiche, gestendo le criticità in itinere per garantire la stabilità del progetto.
6. **Review and evaluation:** consiste negli incontri periodici con il tutor aziendale per mostrare avanzamenti e ricevere feedback. Vengono applicati processi di **Verifica_G** e **Validazione_G** per dimostrare che il software soddisfi i requisiti e sia pronto per l'accettazione finale.

7. **Delivery and completion:** fase conclusiva di consegna del prodotto e della documentazione. Include il supporto durante i test di accettazione e l'eventuale formazione del personale aziendale per l'installazione e l'utilizzo del software, secondo i termini contrattuali stabiliti.

2.1.3 Documentazione fornita

Il gruppo **The Bitless** si impegna nella redazione e nel costante aggiornamento della documentazione di progetto. Tali documenti sono essenziali per garantire la continuità operativa del team, mantenere la coerenza metodologica e assicurare una comunicazione trasparente con tutti i soggetti coinvolti.

- **Lettera di Candidatura:** istanza formale con cui il gruppo manifesta l'interesse per il capitolato C6 proposto da **Zucchetti S.p.A.**. Illustra le ragioni della scelta e fornisce le coordinate per accedere ai repository_G ufficiali del progetto.
- **Analisi dei Capitolati:** documento di analisi comparativa che espone le motivazioni tecniche e logistiche che hanno portato alla selezione del progetto **Second Brain** rispetto alle altre proposte disponibili. Contiene l'analisi in dettaglio di ogni capitolato, descrivendone l'obiettivo, le tecnologie adottate e le considerazioni emerse durante il confronto nel gruppo.
- **Dichiarazione degli impegni:** documento che ufficializza l'impegno operativo dei membri del team, stabilito in un monte ore produttivo di 93 ore per ciascun componente. Include il piano economico e la strategia di rotazione dei ruoli_G ogni due settimane, volta a garantire un'esperienza formativa completa e un carico di lavoro bilanciato tra i componenti.
- **Lettera di Presentazione:** documento tramite il quale il gruppo intende formalizzare la propria candidatura alle revisioni di avanzamento legate alle *baseline_G* del progetto didattico, ovvero la *Requirements and Technology Baseline* (RTB) e la *Product Baseline* (PB).
- **Analisi dei Requisiti:** descrive in modo dettagliato le funzionalità e i vincoli del sistema **Second Brain** (Capitolato C6). L'identificazione delle necessità avviene tramite lo studio dei **Casi d'Uso_G**, corredati da diagrammi illustrativi e matrici di tracciamento_G per assicurare la totale copertura dei requisiti concordati.
- **Norme di Progetto:** documento cardine che definisce il *Way of Working_G* del gruppo sulla base dello standard **ISO/IEC 12207:1995**. Al suo interno sono fissate le procedure per ogni attività, le convenzioni per scrivere il codice, gli strumenti da utilizzare e le tecniche per migliorare costantemente la qualità del lavoro prodotto.
- **Piano di Progetto:** elaborato di pianificazione strategica gestito con metodologia incrementale_G. Analizza la ripartizione temporale delle attività, la gestione delle risorse e dei rischi, fornendo un monitoraggio costante dell'avanzamento e del bilancio dei costi_G sostenuti.
- **Piano di Qualifica:** specifica i protocolli di **Verifica** e **Validazione** adottati dal team. Contiene il dettaglio dei test eseguiti sul prodotto, le metriche di qualità applicate e i resoconti dei risultati ottenuti per garantire la conformità agli standard prefissati.

- **Glossario:** dizionario condiviso creato per raccogliere e spiegare i termini tecnici legati allo specifico dominio_G del progetto. Il suo scopo è evitare malintesi, assicurando che ogni parola abbia un unico significato per tutti i membri; i termini presenti in questa lista sono contrassegnati nei documenti dal pedice "G".
- **Verbali (Interni ed Esterni):** documenti che contengono il riassunto delle discussioni e l'elenco delle decisioni prese durante le riunioni. I verbali interni tracciano l'evoluzione del progetto tra i componenti di **The Bitless**, mentre quelli esterni cristallizzano feedback e scadenze concordate con il committente_G.

2.1.4 Procedure Operative

Il gruppo **The Bitless** adotta le seguenti procedure per garantire il corretto svolgimento dell'attività:

1. Comunicazione:

- **Canale Formale:** Ogni comunicazione e richiesta di chiarimento deve avvenire esclusivamente tramite l'indirizzo email dedicato fornito da Zucchetti. Questo garantisce la tracciabilità e la storicità delle decisioni.

2. Gestione e Pianificazione degli Incontri SAL:

- **Frequenza e Necessità:** Gli incontri di Stato Avanzamento Lavori (SAL) non sono prefissati rigidamente, ma vengono organizzati ogniqualvolta si presenti la necessità di validare una milestone o discutere criticità emergenti.
- **Preavviso Minimo:** Per rispettare la disponibilità del referente aziendale e permettere al team una preparazione adeguata, ogni incontro deve essere richiesto con un preavviso minimo di **48 ore**.
- **Preparazione del Responsabile:** Prima di ogni incontro, il Responsabile di turno ha l'obbligo di redigere una sintesi dettagliata che riporti gli obiettivi raggiunti e le eventuali difficoltà tecniche o organizzative riscontrate.

3. Verbalizzazione:

- **Redazione:** Al termine di ogni sessione con il cliente, deve essere prodotto un **Verbale Esterno**, che riporterà la data e ora dell'incontro, i partecipanti, gli argomenti trattati e le decisioni prese. Il documento serve a cristallizzare gli accordi presi e a evitare ambiguità future.
- **Revisione Interna:** Prima dell'archiviazione finale, il verbale deve essere sottoposto a una fase di *peer-review* interna: un membro del gruppo diverso dall'estensore deve verificarne la correttezza dei contenuti e la forma.

2.1.5 Strumenti

Per garantire un coordinamento efficace delle attività di fornitura, sono stati selezionati i seguenti strumenti:

- **GitHub_G**: utilizzato come infrastruttura principale per la gestione del progetto. Nello specifico:
 - **GitHub Issues**: impiegato per la gestione del *Backlog_G* e del sistema di *ticketing_G*, permettendo di tracciare ogni singola unità di lavoro, assegnare responsabilità e segnalare anomalie;
 - **GitHub Projects**: utilizzato per la pianificazione e il monitoraggio dell'avanzamento globale, facilitando l'organizzazione dei task tramite *Kanban board*.
- **Discord e WhatsApp**: canali dedicati rispettivamente alle riunioni sincrone e alle comunicazioni rapide asincrone tra i membri del gruppo;
- **Google Mail**: canale ufficiale per la corrispondenza formale. Viene impiegato esclusivamente per le comunicazioni scritte verso i referenti, garantendo piena tracciabilità;
- **Google Meet**: piattaforma scelta per le conferenze da remoto con l'azienda proponente. Questo strumento permette di svolgere revisioni periodiche e presentazioni dello stato di avanzamento in un ambiente stabile e condiviso.

2.2 Sviluppo

2.2.1 Processo di sviluppo

Il gruppo adotta lo standard **ISO/IEC 12207:1995** come riferimento per l'intero ciclo di vita del software. Tale norma fornisce le linee guida per il processo di sviluppo, articolato nelle attività fondamentali di analisi, progettazione, codifica, integrazione_G, test, installazione e accettazione. La conduzione di ciascuna attività è vincolata al rispetto delle indicazioni contenute nel contratto con il committente, al fine di garantire che l'implementazione finale sia pienamente conforme ai requisiti e alle specifiche concordate.

2.2.2 Attività previste

In conformità alla subclause 5.3 dello standard, il processo di sviluppo si articola nelle seguenti tredici attività:

1. **Process implementation**: definizione dei piani operativi, selezione degli strumenti tecnologici, dei linguaggi di programmazione e delle metodologie di sviluppo da adottare lungo l'intero ciclo di vita del progetto.
2. **System requirements analysis**: analisi del contesto d'uso e dei bisogni della proponente, finalizzata alla derivazione dei **requisiti funzionali_G**, di capacità e di sicurezza del sistema. I requisiti vengono tradotti in specifiche tecniche documentate, verificabili e tracciabili.
3. **System architectural design**: definizione dell'architettura di sistema ad alto livello, con identificazione delle componenti software e hardware, delle interfacce esterne e delle dipendenze inter-modulo. In questa fase si stabilisce la topologia generale del sistema.

4. **Software requirements analysis:** raffinamento delle specifiche di ogni singola componente software, comprendente la definizione delle interfacce utente, dei contratti di interfaccia e dei **requisiti di qualificazione_G** necessari a validare il corretto funzionamento di ciascun modulo.
5. **Software architectural design:** trasformazione dei requisiti software in una struttura gerarchica di moduli. In questa attività si producono il design preliminare delle interfacce applicative, lo schema logico del **database_G** e la distribuzione delle responsabilità tra i layer architetturali.
6. **Software detailed design:** scomposizione delle componenti architetturali in **unità software_G** elementari. Viene completato il design di dettaglio — comprensivo di diagrammi UML delle classi, sequenze e stati — in modo da rendere ogni unità direttamente codificabile in modo indipendente.
7. **Software coding and testing:** realizzazione del codice sorgente e delle strutture dati. Ogni sviluppatore esegue i **test unitari_G** sulla propria unità per verificarne la conformità alle specifiche di dettaglio prima dell'integrazione.
8. **Software integration:** assemblaggio progressivo dei moduli sviluppati individualmente secondo un piano di **integrazione_G** concordato. I componenti aggregati vengono sottoposti a test di integrazione per verificare la correttezza del flusso dati e la coerenza delle interfacce inter-modulo.
9. **Software qualification testing:** esecuzione di test formali sull'intero sistema software integrato per dimostrarne la conformità all'insieme dei requisiti software stabiliti nella fase di analisi.
10. **System integration:** unione del software con i sistemi hardware e le dipendenze esterne. Il sistema complessivo viene testato durante l'assemblaggio per verificarne la coerenza architetturale globale.
11. **System qualification testing:** collaudo finale del sistema integrato nella sua interezza, finalizzato a certificare la piena soddisfazione dei requisiti di sistema originali e la prontezza del prodotto per la consegna.
12. **Software installation:** deployment del prodotto nell'ambiente di destinazione. Si verifica che il codice, la configurazione e le basi di dati siano inizializzati correttamente e che il sistema operi come atteso nell'ambiente target.
13. **Software acceptance support:** supporto tecnico al committente durante i test di accettazione formale, con l'obiettivo di attestare la conformità del sistema consegnato ai requisiti contrattualmente definiti.

2.2.3 Procedure operative

2.2.3.1 Analisi dei Requisiti

2.2.3.1.1 Descrizione

L'analisi dei **requisiti_G** è un'attività_G critica nell'ambito dello sviluppo software_G, poiché stabilisce le fondamenta per il design, l'implementazione e i test_G del sistema_G. Lo scopo dell'analisi dei requisiti_G è comprendere e definire in modo esaustivo le esigenze del cliente e del

sistema_G, traducendole in specifiche tecniche verificabili e tracciabili. L'attività_G di analisi richiede di rispondere a domande fondamentali: “Qual è il dominio applicativo?”, “Quali sono le reali necessità della proponente?”, “Cosa deve fare il sistema per soddisfarle?”. Essa consiste nella comprensione approfondita del dominio e nella definizione chiara di obiettivi, vincoli e requisiti sia tecnici che funzionali, derivati dall'interazione sistematica con la proponente e dall'analisi dei casi d'uso_G.

2.2.3.1.2 Obiettivi

- Definire insieme alla proponente_G gli obiettivi del prodotto per rispecchiarne le aspettative, tramite identificazione, documentazione e validazione dei requisiti funzionali e non funzionali;
- Facilitare la comprensione comune tra gli stakeholder_G coinvolti nel progetto, riducendo ambiguità e disallineamenti interpretativi;
- Fornire ai progettisti_G requisiti chiari, atomici e di facile comprensione, sui quali fondare le successive scelte architetture;
- Permettere una stima realistica delle tempistiche e dei costi di sviluppo;
- Favorire le attività_G di verifica e test_G fornendo riferimenti pratici e criteri di accettazione oggettivi.

2.2.3.1.3 Documentazione

È compito degli **Analisti**_G effettuare l'analisi dei requisiti_G, redigendo il documento *Analisi dei Requisiti*, che deve contenere le seguenti sezioni:

- **Introduzione:** finalità, sviluppo e riferimenti del documento;
- **Descrizione del prodotto:** analisi del prodotto, articolata in:
 - prospettiva del prodotto;
 - obiettivi del prodotto;
 - funzionalità principali;
 - utenza di riferimento.
- **Casi d'uso:** funzionalità offerte dal sistema_G dal punto di vista dell'utente, comprensivi di:
 - Definizione degli attori coinvolti;
 - Elenco dei casi d'uso in formato testuale strutturato;
 - Diagrammi UML_G dei casi d'uso.
- **Requisiti:** scomposizione delle specifiche in:
 - Requisiti funzionali;
 - Requisiti qualitativi;
 - Requisiti di vincolo.

2.2.3.2 Casi d'uso

I casi d'uso forniscono una descrizione dettagliata delle funzionalità del sistema_G dal punto di vista degli utenti, identificando come il sistema_G risponde a determinate azioni o scenari. Sono strumenti utilizzati nell'analisi dei requisiti_G per catturare e illustrare in modo chiaro e comprensibile le interazioni tra gli utenti e il software_G, e i relativi risultati attesi. Ogni caso d'uso testuale è costituito dai seguenti elementi:

1. **Identificativo**, nel formato: **UC[Primario].[Secondario] – [Titolo]** dove:
 - **UC**: acronimo di *Use Case*;
 - **[Primario]**: ID_G numerico progressivo del caso d'uso principale;
 - **[Secondario]**: ID_G numerico del sottocaso d'uso, presente esclusivamente per casi d'uso dipendenti dal caso radice;
 - **[Titolo]**: denominazione breve ed esplicativa della funzionalità.
2. **Attore principale**: entità esterna che interagisce attivamente con il sistema_G per soddisfare una propria necessità;
3. **Attore secondario**: eventuale entità esterna che non interagisce direttamente, ma la cui partecipazione è necessaria al sistema_G per soddisfare il bisogno dell'attore principale;
4. **Precondizioni**: stato in cui deve trovarsi il sistema_G affinché la funzionalità sia disponibile all'attore;
5. **Postcondizioni**: stato in cui si trova il sistema_G al termine dell'esecuzione corretta dello scenario principale;
6. **Scenario principale**: sequenza ordinata di eventi che si verificano quando l'attore interagisce con il sistema_G per raggiungere l'obiettivo del caso d'uso;
7. **Scenari alternativi**: sequenze di eventi che si verificano al verificarsi di condizioni specifiche, rappresentando deviazioni dal percorso principale senza raggiungere le postcondizioni attese;
8. **Estensioni**: scenari alternativi che, al verificarsi di condizioni specifiche, deviano il flusso dal percorso principale senza giungere alle postcondizioni attese;
9. **Inclusioni**: scenari che rappresentano funzionalità comuni a più casi d'uso, richiamati obbligatoriamente all'interno del flusso principale per evitare duplicazioni.
10. **Generalizzazioni**: scenari che rappresentano funzionalità condivise tra più casi d'uso, in cui un caso d'uso figlio eredita il comportamento da un caso d'uso padre, consentendo di modellare varianti specifiche senza duplicare il flusso principale.
11. **Trigger**: evento esterno che innesca l'esecuzione del caso d'uso, come un'azione dell'utente o una condizione di sistema.

2.2.3.3 Diagrammi dei casi d'uso

I diagrammi dei casi d'uso sono strumenti grafici che permettono di visualizzare in modo chiaro e intuitivo le funzionalità offerte dal sistema_G dal punto di vista dell'utente. Consentono di identificare rapidamente le relazioni tra i vari casi d'uso e offrono una visione d'insieme delle interazioni tra attori e sistema_G, senza approfondire dettagli implementativi. Di seguito sono descritti i principali componenti di un diagramma dei casi d'uso.

- **Attori:** entità esterne al sistema_G che interagiscono con esso (utenti umani, altri sistemi software_G o componenti esterne). Sono rappresentati come *stickman* posizionati all'esterno del rettangolo che delimita il sistema_G.
- **Casi d'uso:** funzionalità offerte dal sistema_G con cui un attore può interagire. Ogni caso d'uso è rappresentato da un'ellisse contenente il proprio identificativo e un titolo esplicativo.
- **Sistema:** rappresentato da un rettangolo con titolo identificativo. Al suo interno sono collocati i casi d'uso; all'esterno, gli attori.

Relazioni tra attori e casi d'uso:

- **Associazione:** linea continua che collega un attore a un caso d'uso, indicando il coinvolgimento dell'attore nell'interazione descritta.

Relazioni tra attori:

- **Generalizzazione tra attori:** relazione di ereditarietà in cui un attore figlio eredita comportamenti e caratteristiche dall'attore padre. Rappresentata con una linea continua e freccia vuota dall'attore figlio all'attore padre.

Relazioni tra casi d'uso:

- **Inclusione («include»):** indica che un caso d'uso includente incorpora obbligatoriamente l'esecuzione di un caso d'uso incluso. La relazione è rappresentata da una freccia tratteggiata orientata dal caso d'uso includente verso quello incluso. È utile per modellare comportamenti comuni a più casi d'uso, evitando duplicazioni.
- **Estensione («extend»):** indica che un caso d'uso estendente può ampliare il comportamento di un caso d'uso base al verificarsi di condizioni specifiche. Rappresentata da una freccia tratteggiata orientata dal caso d'uso estendente verso quello esteso. Consente di gestire scenari alternativi senza appesantire il flusso principale.
- **Generalizzazione tra casi d'uso:** relazione di ereditarietà in cui un caso d'uso specifico eredita il comportamento da un caso d'uso più generico. Rappresentata con una linea continua e freccia vuota dal caso d'uso figlio al caso d'uso padre.

2.2.3.4 Requisiti

I requisiti di un prodotto software_G sono specifiche dettagliate e documentate che delineano le funzionalità, le prestazioni, i vincoli e gli altri aspetti critici che il software_G deve soddisfare. Fungono da guida per lo sviluppo, il testing e la valutazione del prodotto, assicurando che il sistema_G risponda alle esigenze degli utenti e agli obiettivi del progetto. I requisiti si distinguono in:

- **Requisiti funzionali:** descrivono le funzionalità che il software_G deve possedere, ovvero i comportamenti attesi del sistema_G in risposta a input o condizioni specifiche;
- **Requisiti qualitativi:** definiscono criteri di prestazione, qualità, usabilità e altri attributi non direttamente legati alle funzionalità specifiche del sistema_G;
- **Requisiti di vincolo:** impongono limitazioni tecnologiche, normative o architetturali entro cui il sistema_G deve operare.

Ogni requisito è costituito dai seguenti elementi:

1. **Identificativo**, nel formato: **R[Tipologia][Codice]** dove:

- **Tipologia:**
 - **RF** — Requisito Funzionale;
 - **RQ** — Requisito Qualitativo;
 - **RV** — Requisito di Vincolo.
- **Codice:** identificativo numerico progressivo univoco all'interno della tipologia di appartenenza.

2. **Priorità:**

- **Obbligatorio:** requisito irrinunciabile per uno o più stakeholder_G; il mancato soddisfacimento pregiudica il rilascio del prodotto;
- **Desiderabile:** non strettamente necessario al rilascio, ma apporta valore aggiunto significativo;
- **Opzionale:** requisito accessorio, implementabile subordinatamente alla disponibilità di risorse o negoziabile in fasi successive.

3. **Descrizione:** narrazione chiara, univoca e dettagliata del comportamento o della caratteristica che il software_G deve possedere, priva di ambiguità interpretative;

4. **Fonte:** origine del requisito, ad esempio: Capitolato, Verbale interno, Verbale esterno;

5. **Casi d'uso associati:** elenco degli identificativi UC correlati al requisito.

2.2.3.5 Progettazione

2.2.3.5.1 Descrizione

Lo scopo primario dell'attività_G di progettazione consiste nell'identificare una soluzione realizzativa ottimale che soddisfi appieno le esigenze di tutti gli stakeholder_G, nel rispetto dei requisiti definiti e delle risorse disponibili. La progettazione risponde alla domanda chiave: *“Qual è il modo migliore per realizzare ciò di cui c'è bisogno?”*.

È fondamentale definire l'architettura_G del prodotto prima di avviare la fase di codifica, perseguendo una **correttezza per costruzione** piuttosto che una correttezza per correzione. Questo approccio permette di gestire in modo efficiente la complessità del prodotto, garantendo una struttura robusta e coesa lungo l'intero ciclo di vita del sistema_G.

Il processo si articola in due livelli distinti e sequenziali:

- **Progettazione architetturale:** definizione della struttura ad alto livello del sistema_G, con identificazione dei macro-componenti, delle loro responsabilità e delle interfacce tra essi;
- **Progettazione di dettaglio:** scomposizione delle componenti architetture in unità software_G elementari, sufficientemente dettagliate da essere codificate in modo indipendente.

2.2.3.5.2 Obiettivi

L'obiettivo principale è assicurare il soddisfacimento dei requisiti attraverso un sistema_G di qualità, delineato dall'architettura_G del prodotto. Ciò comporta:

- Identificare parti componibili conformi ai requisiti, dotate di specifiche chiare e coese, svilupparli con risorse sostenibili e costi contenuti;
- Organizzare il sistema_G in modo da agevolare adattamenti e estensioni future, minimizzando l'impatto delle modifiche sulle componenti esistenti;
- Gestire la complessità del sistema_G suddividendolo in unità architetture indipendenti, ciascuna semplice da implementare, verificare e mantenere;
- Selezionare con attenzione le tecnologie più adatte, valutandone punti di forza, debolezze ed eventuali criticità prima di procedere con l'implementazione;
- Fornire ai programmatori_G istruzioni progettuali precise, riducendo ambiguità e disallineamenti durante la fase di codifica.

Il flusso operativo della progettazione prevede: un'analisi tecnologica preliminare, la definizione di un'architettura_G di alto livello che costituisce la base per il **Proof of Concept** (PoC_G), la valutazione del PoC_G e il successivo raffinamento iterativo fino al design completo. Tale design definitivo guiderà lo sviluppo del **Minimum Viable Product** (MVP_G), che costituirà la versione funzionale essenziale del prodotto inclusa nella Product Baseline.

2.2.3.5.3 Documentazione

Specifica Tecnica

Il documento di *Specifica Tecnica* descrive in modo approfondito il design definitivo del prodotto e fornisce istruzioni precise agli sviluppatori, guidandoli nella corretta implementazione della soluzione software_G secondo i requisiti e le specifiche indicate. Questo documento comprende i seguenti elementi:

- **Tecnologie utilizzate:** specifica delle tecnologie, dei framework_G e delle librerie_G di terze parti integrate nel sistema_G, con motivazione delle scelte effettuate;
- **Architettura logica:** definizione dei componenti del sistema_G, dei loro ruoli, delle connessioni e delle modalità di interazione;
- **Architettura di deployment:** descrizione di come le componenti architetturali vengono allocate e distribuite nell'ambiente di esecuzione;
- **Pattern architetturali e design pattern_G:** descrizione dei pattern adottati, con motivazione delle scelte e rappresentazione grafica del loro funzionamento nel contesto del progetto;
- **Schema del database_G:** modello logico delle entità, con attributi, chiavi primarie, chiavi esterne e vincoli di integrità referenziale;
- **Vincoli e linee guida:** restrizioni e regole progettuali da rispettare durante lo sviluppo;
- **Requisiti tecnici:** specifiche prestazionali, di sicurezza, di scalabilità e di compatibilità che il sistema_G deve soddisfare.

2.2.3.6 Qualità dell'architettura

L'architettura_G prodotta deve soddisfare i seguenti attributi qualitativi:

- **Sufficienza:** soddisfazione di tutti i requisiti funzionali e non funzionali definiti, senza sovradimensionamento né sottodimensionamento;
- **Comprensibilità:** chiarezza strutturale tale da permettere a sviluppatori e stakeholder_G di comprendere il funzionamento del sistema_G senza ambiguità;
- **Modularità:** suddivisione in moduli o componenti chiaramente definiti, con separazione netta delle responsabilità, per facilitare manutenzione e sviluppo parallelo;
- **Robustezza:** capacità di gestire situazioni anomale o errate senza interruzioni gravi o perdite di dati;
- **Flessibilità:** facilità di adattamento o estensione del sistema_G a fronte di nuovi requisiti, senza richiedere modifiche strutturali radicali;
- **Riusabilità:** capacità di riutilizzare componenti in contesti diversi, riducendo lo sforzo complessivo di sviluppo;
- **Efficienza:** utilizzo ottimale delle risorse disponibili (memoria, CPU, rete) per l'esecuzione delle funzionalità nel minor tempo possibile;

- **Affidabilità:** capacità del sistema_G di svolgere correttamente le proprie funzioni in modo coerente e prevedibile nel tempo;
- **Sicurezza rispetto a malfunzionamenti (Safety):** prevenzione di danni a persone, beni o dati causati da malfunzionamenti del software_G;
- **Sicurezza rispetto a intrusioni (Security):** protezione del sistema_G da accessi non autorizzati, manipolazioni e intrusioni esterne;
- **Semplicità:** mantenimento dell'architettura_G il più semplice possibile senza compromettere funzionalità o efficacia;
- **Incapsulazione:** occultamento dei dettagli implementativi all'esterno di ciascuna componente, con accesso consentito esclusivamente tramite interfacce definite;
- **Coesione:** elevata correlazione tra gli elementi interni di ciascun modulo, che devono collaborare verso un obiettivo comune ben definito;
- **Basso accoppiamento:** minimizzazione delle dipendenze tra moduli distinti, per migliorare manutenibilità, testabilità e flessibilità del sistema_G.

2.2.3.7 Diagrammi UML

I diagrammi UML_G costituiscono lo strumento primario di documentazione progettuale. I principali vantaggi del loro utilizzo sono:

- **Chiarezza nella comunicazione:** rendono comprensibili i concetti tecnici a team e stakeholder_G, indipendentemente dal loro livello di dettaglio implementativo;
- **Standardizzazione:** offrono un linguaggio comune e non ambiguo per la documentazione dei sistemi software_G;
- **Analisi visiva:** aiutano a identificare errori strutturali e lacune progettuali prima della fase di implementazione, riducendo il costo delle correzioni;
- **Supporto alla manutenzione:** semplificano la comprensione della struttura del software_G da parte di sviluppatori che intervengono in fasi successive.

A supporto dell'attività_G di progettazione vengono utilizzati i seguenti tipi di diagramma.

Diagrammi delle classi. Ciascun diagramma delle classi rappresenta le proprietà e le relazioni tra le componenti del sistema_G in una prospettiva statica, indipendente dall'esecuzione. Le classi sono rappresentate graficamente come rettangoli suddivisi in tre sezioni:

1. **Nome della classe:** identificativo in grassetto, secondo la convenzione *PascalCase*, che deve esplicitare chiaramente la responsabilità della classe. Se la classe è astratta, il nome è scritto in corsivo;
2. **Attributi:** ciascun attributo occupa una riga separata, nel formato: **Visibilità Nome: Tipo [Molteplicità] = ValoreDefault** dove:
 - **Visibilità:** - privata, + pubblica, # protetta, ~ di package;

- **Nome:** identificativo rappresentativo, in notazione *camelCase*; se costante, in *UPPER_SNAKE_CASE*;
 - **Molteplicità:** per sequenze di elementi, espressa come `tipo[n]` o `tipo[*]` se la cardinalità non è nota a priori;
 - **Valore di default:** opzionale, specificato ove applicabile.
3. **Firme dei metodi:** ciascun metodo occupa una riga separata, nel formato: **Visibilità** **Nome**(Parametri): **TipoRitorno** dove:
- **Visibilità:** segue la convenzione definita per gli attributi;
 - **Nome:** identificativo univoco in *PascalCase*, descrittivo dell'obiettivo del metodo;
 - **Parametri:** da 0 a n , separati da virgola, con la stessa notazione degli attributi;
 - **Tipo di ritorno:** tipo del valore restituito.

Convenzioni aggiuntive sui metodi:

- I metodi *getter*, *setter* e i costruttori non vengono inclusi tra i metodi rappresentati;
- I metodi astratti sono scritti in corsivo;
- I metodi statici sono sottolineati;
- L'assenza di attributi o metodi determina la visualizzazione della sezione corrispondente come campo vuoto.

Relazioni tra le classi.

- **Dipendenza:** la classe A utilizza servizi o membri della classe B. Rappresentata con una freccia tratteggiata da A verso B. Indica il minor grado di accoppiamento: un cambiamento in B può influenzare A, ma non viceversa;
- **Associazione:** la classe A contiene campi o istanze della classe B. Rappresentata con una linea continua orientata. Le molteplicità sono espresse agli estremi della freccia: $0..1$, $0..*$, 1 , $*$, n ;
- **Aggregazione:** relazione "parte di" (*part-of*) in cui una classe è composta da altre classi, che tuttavia possono esistere indipendentemente dalla classe principale. Rappresentata con una freccia a diamante vuoto sul lato dell'aggregante;
- **Composizione:** forma più forte di aggregazione in cui le parti non possono esistere indipendentemente dalla classe principale. Rappresentata con un diamante pieno sul lato del componente principale;
- **Generalizzazione:** relazione di ereditarietà tra classi. Rappresentata con una linea continua e freccia vuota dalla classe figlia alla classe padre.

Diagrammi di sequenza. I diagrammi di sequenza rappresentano la dimensione dinamica del sistema_G, descrivendo le interazioni tra oggetti in funzione del tempo per i casi d'uso principali. Consentono di verificare la correttezza dei flussi di comunicazione inter-modulo prima della fase di codifica.

2.2.3.8 Design Pattern

I design pattern_G costituiscono risposte consolidate a problemi di progettazione ricorrenti in determinati contesti. Offrono un approccio riusabile che garantisce qualità della soluzione e riduce i tempi di implementazione. L'adozione di un pattern_G avviene quando una soluzione si è dimostrata efficace in un contesto analogo a quello corrente.

Ogni design pattern_G adottato deve essere documentato con:

- Una **rappresentazione grafica** che ne illustri il funzionamento nel contesto specifico del progetto;
- Una **spiegazione testuale** della logica del pattern_G e del problema che risolve;
- Una **descrizione della sua utilità** all'interno dell'architettura_G complessiva del sistema_G.

2.2.3.9 Codifica

2.2.3.9.1 Descrizione

L'attività_G di codifica è affidata al ruolo del **Programmatore_G** e rappresenta il momento in cui le funzionalità definite dalla proponente_G e progettate dai **Progettisti_G** prendono forma concreta. Durante questa fase, i modelli architetturali e i diagrammi di dettaglio vengono tradotti in codice sorgente eseguibile, creando le istruzioni e le strutture dati che il calcolatore interpreterà ed eseguirà.

I Programmatore_G devono rispettare scrupolosamente le linee guida e le norme stabilite dal gruppo, al fine di garantire che il codice prodotto sia coerente, manutenibile e verificabile nel tempo, indipendentemente da chi lo ha scritto.

2.2.3.9.2 Obiettivi

La codifica è finalizzata alla creazione di un prodotto software_G conforme alle richieste del committente_G e agli accordi contrattuali stipulati. Il rigoroso rispetto delle norme di codifica garantisce:

- La creazione di codice di alta qualità, chiaro e privo di ambiguità;
- La facilità di manutenzione, estensione e refactoring nel tempo;
- La semplificazione delle attività_G di verifica e test_G, tramite l'adozione di strutture modulari e predisposte all'isolamento;
- L'uniformità stilistica e tecnica tra i moduli sviluppati da componenti diversi del team.

2.2.3.10 Norme di codifica

Le seguenti norme sono state formalizzate prendendo spunto dal libro "*Clean Code*" di Robert C. Martin e dalle best practice_G consolidate per i linguaggi adottati nel progetto.

- **Nomi significativi:** utilizzare nomi che riflettano il significato e lo scopo delle variabili, funzioni e classi. Evitare abbreviazioni ambigue o identificatori generici come `tmp`, `data`, `obj`. I nomi devono essere autoesplicativi e non richiedere commenti aggiuntivi per essere compresi;
- **Lingua:** identificatori, commenti e documentazione inline devono essere redatti in lingua inglese, per favorire uniformità e compatibilità con strumenti di analisi statica internazionali;
- **Convenzioni di nomenclatura:**
 - *camelCase* per variabili e funzioni in React/JavaScript_G;
 - *snake_case* per variabili, funzioni e nomi di moduli in Python_G;
 - *PascalCase* per classi in Python_G e componenti React_G;
 - *UPPER_SNAKE_CASE* per costanti globali, variabili d'ambiente e costanti di modulo;
 - *kebab-case* per i nomi dei file dei componenti React_G (es. `my-component.jsx`).
- **Indentazione e formattazione:** utilizzo di 4 spazi per l'indentazione in Python_G, 2 spazi per JavaScript/React_G, in conformità con PEP 8 e le best practice React. La formattazione automatica del codice è delegata agli strumenti **black** (per Python_G) e **Prettier** (per React_G), la cui configurazione è condivisa nel repository;
- **Lunghezza delle righe:** le righe di codice non devono superare i **120 caratteri**. Il rispetto di tale limite favorisce la leggibilità su schermi di diverse dimensioni e la visualizzazione affiancata di più file;
- **Lunghezza e responsabilità dei metodi:** ogni funzione o metodo deve svolgere una **singola responsabilità ben definita**, in conformità al principio *Single Responsibility Principle* (SRP_G) dei principi SOLID_G. La complessità ciclomatica di ogni funzione non deve superare il valore di **10**; qualora superata, il codice deve essere refactorizzato in unità più piccole. Questo favorisce:
 - Chiarezza e leggibilità;
 - Manutenibilità e comprensibilità;
 - Testabilità: funzioni brevi sono più semplici da isolare nei test unitari_G.
- **Documentazione inline:** ogni funzione pubblica, classe e modulo deve essere documentato con commenti nel formato standard del linguaggio adottato:
 - **docstring** in formato Google style per i moduli, le classi e le funzioni in Python_G;
 - **JSDoc** per le funzioni e i componenti React_G.

I commenti devono descrivere il *perché* di una scelta implementativa non ovvia, non il *cosa* fa il codice, che deve essere autoesplicativo;

- **Gestione degli errori:** in Python_G, le eccezioni vanno gestite esplicitamente con blocchi `try-except`, evitando la cattura generica di `Exception` senza un'adeguata registrazione o gestione. In React_G, le eccezioni devono essere intercettate tramite componenti `Error Boundary` e le operazioni asincrone gestite con `.catch()` o blocchi `try-catch` all'interno di funzioni `async`;

- **Test unitari_G**: ogni unità software_G prodotta deve essere accompagnata dai relativi test unitari_G. La copertura minima dei rami logici richiesta è dell'**80%**. I test devono essere indipendenti, ripetibili e privi di dipendenze da stato globale o da risorse esterne non mockate. Per Python si utilizza **pytest**, per React si impiegano **Jest** e **React Testing Library**;
- **Assenza di codice commentato**: il codice non più utilizzato deve essere eliminato e non lasciato commentato nel sorgente. Il versionamento tramite Git_G garantisce la possibilità di recuperare versioni precedenti senza necessità di conservare codice morto.

2.2.3.11 Configurazione Ambiente di esecuzione

L'ambiente di esecuzione del sistema è progettato per garantire riproducibilità, isolamento e facilità di deployment. Viene adottata la containerizzazione mediante **Docker Compose**: il backend Python e il server Nginx che serve il frontend React compilato risiedono in container separati. Le dipendenze sono dichiarate nei file `requirements.txt` e `package.json`; le variabili d'ambiente sono iniettate a runtime e mai incluse nelle immagini. In fase di sviluppo, volumi montati abilitano il live reload, riducendo i tempi di feedback. L'infrastruttura riproduce fedelmente l'ambiente di produzione, assicurando coerenza durante tutto il ciclo di vita.

2.2.4 Strumenti a supporto

Il team si avvale dei seguenti strumenti, organizzati per categoria, per supportare le attività di sviluppo, verifica e gestione del progetto.

- **Modellazione UML**: **StarUML**, per la creazione di diagrammi dei casi d'uso, delle classi, di sequenza e dei package.
- **Ambiente di sviluppo**: **Visual Studio Code**, corredato delle estensioni specifiche per React e Python.
- **Qualità del codice**: **ESLint** e **Prettier** per il frontend React/TypeScript; **black** per la formattazione automatica del codice Python.
- **Testing**: **Vitest**, **React Testing Library** e **Playwright** per il frontend; **pytest** per il backend.
- **Containerizzazione**: **Docker** e **Docker Compose** per la creazione di ambienti riproducibili e isolati.
- **Integrazione continua**: **GitHub Actions**, configurata per eseguire lint, type checking, test, verifica della copertura e build a ogni pull request.
- **Versionamento**: **Git**, con repository mono-repo organizzato in `/web` e `/api`.

3 Processi di supporto

I processi_G di supporto comprendono le attività trasversali necessarie a garantire il corretto svolgimento dei processi primari e organizzativi, assicurando che ogni artefatto prodotto sia conforme agli standard di qualità prefissati.

Tra i processi_G di supporto applicati nel progetto si distinguono:

3.1 Documentazione

3.2 Documentazione

3.2.1 Processo

La documentazione rappresenta uno degli elementi fondamentali per il corretto governo del progetto, poiché consente di formalizzare e mantenere tracciabili decisioni progettuali, requisiti, attività svolte, specifiche tecniche e stato di avanzamento del lavoro.

Tutti i documenti prodotti dal gruppo costituiscono artefatti_G ufficiali del progetto e devono quindi rispettare:

- **standard qualitativi condivisi:** ogni documento deve raggiungere un livello minimo di qualità definito nel Piano di Qualifica_G, comprensivo di metriche per la leggibilità, completezza e correttezza;
- **convenzioni documentali uniformi:** l'intera documentazione adotta lo stesso stile tipografico, la stessa struttura gerarchica e la medesima nomenclatura, in modo da garantire un aspetto coerente e professionale;
- **procedure di verifica rigorose:** prima dell'approvazione ogni documento viene sottoposto a un processo di revisione formale da parte di un Verificatore_G incaricato, che applica le checklist definite nelle presenti norme;
- **criteri di versionamento e tracciabilità:** ogni modifica è registrata in un apposito registro delle modifiche e il documento è identificato da un numero di versione incrementale secondo le regole di semantic versioning_G, consentendo la completa ricostruzione della storia evolutiva;
- **regole di approvazione formale:** nessun documento può essere considerato ufficiale senza l'approvazione esplicita del Responsabile_G, che ne autorizza il consolidamento nel branch principale.

Un processo documentale ben definito permette di:

- **mantenere la tracciabilità delle decisioni progettuali:** ogni scelta architeturale o requisito è collegato a un documento che ne spiega le motivazioni, facilitando future analisi di impatto;
- **garantire coerenza tra stato reale del progetto e documentazione ufficiale:** la documentazione viene aggiornata a ogni modifica significativa del prodotto, in modo da rispecchiare fedelmente lo stato corrente del sistema e ridurre il debito informativo;
- **uniformare terminologia e stile redazionale:** l'adozione di un glossario centralizzato e di template comuni evita ambiguità e riduce il carico cognitivo per i lettori;
- **favorire la collaborazione tra i membri del gruppo:** il workflow basato su branch e Pull Request_G consente a più persone di lavorare simultaneamente sugli stessi documenti, gestendo in modo ordinato i conflitti di versione;

- **semplificare le attività di revisione e verifica:** l'uso di strumenti automatici di compilazione e di generazione del PDF permette di verificare immediatamente la resa finale del documento e di concentrare lo sforzo umano sugli aspetti contenutistici;
- **garantire la conservazione storica delle modifiche effettuate:** grazie a Git_G , ogni versione precedente è recuperabile, consentendo confronti e ripristini puntuali.

La documentazione evolve parallelamente all'avanzamento del progetto e viene consolidata progressivamente in corrispondenza delle baseline_G previste. Una baseline_G congela uno stato stabile della documentazione, che funge da riferimento per le successive fasi di sviluppo e verifica.

Tra i principali documenti prodotti dal gruppo rientrano:

- **Norme di Progetto $_G$ (NdP):** documento che definisce le regole, le procedure e gli strumenti adottati dal gruppo per tutte le attività di progetto;
- **Analisi dei Requisiti $_G$ (AdR):** documento che raccoglie, classifica e formalizza i requisiti del sistema, distinguendo tra requisiti funzionali, di qualità e di vincolo;
- **Piano di Progetto $_G$ (PdP):** documento che descrive la pianificazione temporale, l'allocazione delle risorse e l'organizzazione del lavoro;
- **Piano di Qualifica $_G$ (PdQ):** documento che definisce le metriche di qualità, le procedure di misurazione e gli obiettivi qualitativi del prodotto e dei processi;
- **documentazione tecnica e progettuale:** include diagrammi UML, specifiche di interfaccia, tracciati record e documentazione delle API;
- **verbali interni:** registrazioni formali delle riunioni interne del gruppo, contenenti decisioni, assegnazioni e scadenze;
- **verbali esterni:** resoconti degli incontri con l'azienda proponente, utili per cristallizzare accordi e requisiti.

Ogni documento deve rappresentare fedelmente lo stato corrente del progetto e deve essere mantenuto costantemente aggiornato. In caso di discrepanza tra il prodotto software e la documentazione, quest'ultima deve essere tempestivamente corretta per ripristinare la coerenza informativa.

3.2.1.1 Documentation as Code

Il gruppo adotta il paradigma *Documentation as Code*, secondo il quale la documentazione viene trattata con le stesse metodologie, procedure e strumenti utilizzati per il codice sorgente. La scelta discende dalla volontà di mantenere il massimo allineamento tra lo stato del software e la sua descrizione ufficiale, riducendo al minimo il rischio di documentazione obsoleta.

Questo approccio consente di:

- **mantenere la documentazione sincronizzata con l'evoluzione del progetto:** poiché la documentazione risiede nello stesso repository $_G$ del codice, ogni modifica al prodotto può essere immediatamente accompagnata dall'aggiornamento dei relativi documenti;

- **garantire versionamento e tracciabilità delle modifiche:** la storia di ogni modifica è registrata nei commit_G di Git_G, con autore, data e messaggio descrittivo;
- **applicare workflow_G strutturati di revisione:** le Pull Request_G impongono una revisione obbligatoria prima dell'integrazione, esattamente come per il codice;
- **automatizzare compilazione e pubblicazione dei documenti:** un'apposita pipeline CI/CD compila i sorgenti L^AT_EX_G e rende disponibile il PDF aggiornato nella repository dei documenti, eliminando operazioni manuali e relativi errori;
- **favorire la collaborazione simultanea tra più membri del gruppo:** ogni redattore lavora sul proprio branch e può integrare il lavoro altrui attraverso merge_G non distruttivi.

L'approccio adottato prevede che:

- tutti i documenti siano conservati nella medesima repository_G del progetto, organizzati in cartelle separate per tipologia (verbali, documenti di progetto, documenti tecnici);
- ogni modifica venga effettuata tramite branch_G dedicati, creati a partire dal branch `develop`, con nomi che identificano il tipo di intervento (es. `docs/nome-documento`);
- le modifiche siano sottoposte a Pull Request_G, in cui il redattore descrive il contenuto della modifica e i motivi dell'aggiornamento;
- ogni aggiornamento venga verificato prima dell'integrazione: un verificatore_G esamina il contenuto e accerta il rispetto delle norme prima di autorizzare il merge_G;
- la cronologia delle revisioni sia completamente tracciabile tramite Git_G, permettendo di risalire a ogni singola modifica e di confrontare versioni differenti tramite diff.

L'utilizzo del paradigma *Documentation as Code* consente inoltre di integrare il processo documentale all'interno del configuration management_G del progetto. Di conseguenza, lo stesso strumento di versionamento gestisce sia gli artefatti software sia quelli documentali, garantendo una vista unitaria della configurazione.

3.2.2 Ciclo di vita dei documenti

Il ciclo di vita dei documenti definisce l'insieme degli stati attraversati da ogni artefatto_G documentale durante il proprio utilizzo all'interno del progetto. La formalizzazione del ciclo di vita permette di sapere sempre in quale fase si trova un documento, quali attività sono state completate e quali sono ancora da svolgere.

Ogni documento attraversa progressivamente le seguenti fasi:

1. **Necessità:** emersione di un bisogno informativo che richiede la creazione o l'aggiornamento di un documento;
2. **Pianificazione:** definizione degli obiettivi, della struttura e dell'allocazione delle risorse necessarie per la stesura;
3. **Redazione:** scrittura effettiva del contenuto, seguendo i template e le convenzioni prescritte;

4. **Verifica:** revisione formale da parte di un verifikatore_G indipendente dall'autore;
5. **Approvazione:** validazione finale da parte del Responsabile_G, che sancisce l'ufficialità del documento;
6. **Consolidamento:** archiviazione della versione approvata nel branch `main` e generazione del PDF definitivo;
7. **Manutenzione:** fase continua di aggiornamento a seguito di modifiche progettuali o correzioni.

3.2.2.1 Stato di necessità

Il processo documentale ha origine nel momento in cui emerge la necessità di:

- produrre nuova documentazione, ad esempio quando si avvia una nuova fase del progetto che richiede la stesura del Piano di Qualifica_G o dell'Analisi dei Requisiti_G;
- aggiornare documentazione esistente, per riflettere una modifica architettuale, l'introduzione di nuovi requisiti o la correzione di errori segnalati;
- formalizzare decisioni progettuali, come la scelta di una tecnologia, di un pattern architettuale o di un protocollo di comunicazione, che devono essere registrati per futura consultazione;
- registrare modifiche significative del progetto, come un cambiamento di priorità, la riallocazione di risorse o l'inserimento di nuovi componenti.

La necessità può derivare:

- da obblighi previsti dal capitolato_G, che richiede la consegna di documentazione specifica (es. AdR, PdP, PdQ);
- dall'avanzamento delle attività di sviluppo, che rende necessario documentare nuovi moduli, API o procedure di deploy;
- da richieste dell'azienda proponente, emerse durante incontri o comunicazioni ufficiali;
- da nuove esigenze organizzative, come la riorganizzazione del gruppo o l'introduzione di nuovi ruoli;
- da modifiche ai requisiti, che impattano la documentazione di analisi e di progetto.

3.2.2.2 Pianificazione

Una volta individuata la necessità documentale, il Responsabile_G procede alla pianificazione delle attività, in modo da allocare correttamente le risorse e definire tempi certi per il completamento.

La pianificazione comprende:

1. **identificazione del documento da produrre:** si stabilisce se si tratta di un nuovo documento o di una modifica a uno esistente;

2. **definizione degli obiettivi documentali:** si specifica cosa il documento deve comunicare, a chi è rivolto e con quale livello di dettaglio;
3. **suddivisione del documento in sezioni:** si abbozza l'indice, individuando i capitoli e i paragrafi principali;
4. **definizione delle scadenze:** si fissa una data entro la quale la redazione deve essere completata, in coerenza con la pianificazione di progetto;
5. **assegnazione dei task_G ai membri del gruppo:** il Responsabile_G nomina un redattore (o più redattori per documenti complessi) e un verificatore_G;
6. **apertura delle Issue_G nell'ITS_G adottato:** per ogni attività documentale si crea una issue_G su GitHub, assegnandola al redattore e al verificatore e collegandola alla milestone di competenza.

Durante le riunioni operative il gruppo:

- discute i contenuti da inserire, condividendo conoscenze e allineando le diverse visioni;
- definisce la struttura del documento, decidendo l'ordine delle sezioni e l'eventuale necessità di diagrammi o tabelle;
- stabilisce le convenzioni redazionali, come lo stile delle didascalie, la formattazione del codice e le regole per i riferimenti incrociati;
- organizza la distribuzione del lavoro, suddividendo le sezioni tra più redattori qualora il documento sia particolarmente corposo.

3.2.2.3 Stato di redazione

Durante la fase di redazione il documento viene prodotto secondo le norme definite nelle presenti NdP_G. Il redattore opera sul proprio branch dedicato, scrivendo il contenuto in L^AT_EX_G e rispettando il template fornito.

La redazione deve rispettare:

- **uniformità terminologica:** tutti i termini tecnici devono corrispondere a quelli definiti nel glossario di progetto e non possono essere utilizzati sinonimi ambigui;
- **coerenza stilistica:** il registro linguistico, l'uso dei tempi verbali e la formattazione devono essere omogenei in tutto il documento;
- **correttezza grammaticale:** il testo deve essere privo di errori ortografici e sintattici, anche avvalendosi di correttori automatici;
- **conformità alla struttura documentale definita:** il documento deve seguire il template comune, con frontespizio, registro delle modifiche, indice e corpo strutturato;
- **completezza dei contenuti:** ogni sezione deve essere sviluppata in modo esaustivo, senza lasciare placeholder o frasi incomplete.

Ogni modifica viene effettuata:

- su branch_G dedicati, creati a partire da `develop` con un nome descrittivo (es. `docs/adr-requisiti-funzionali`);
- mediante commit_G atomici e significativi, ciascuno dei quali rappresenta un'unità logica di modifica e reca un messaggio chiaro (es. "Aggiunta sezione requisiti di vincolo");
- mantenendo tracciabilità delle revisioni, grazie all'associazione tra commit_G e issue_G di riferimento, visibile sia su GitHub che nel registro delle modifiche del documento stesso.

3.2.2.4 Stato di verifica

Conclusa la redazione, il documento entra nella fase di verifica formale. Il redattore apre una Pull Request $_G$ verso il branch `develop` e assegna il verificatore $_G$ designato.

La verifica viene effettuata dal Verificatore $_G$, il quale controlla:

- **correttezza grammaticale e sintattica:** revisione attenta del testo per eliminare refusi, errori di battitura e costruzioni sintattiche errate;
- **conformità stilistica:** verifica che il documento rispetti le convenzioni tipografiche (uso corretto di corsivo, grassetto, elenchi, virgolette) definite nel template;
- **coerenza semantica:** controllo che le informazioni riportate siano logicamente coerenti tra loro e con gli altri documenti di progetto, senza contraddizioni;
- **correttezza dei riferimenti:** verifica che tutti i rimandi a sezioni, tabelle, figure e documenti esterni siano funzionanti e corretti;
- **uniformità terminologica:** accertamento che la terminologia sia allineata al glossario e che non vi siano utilizzi difforni dello stesso termine;
- **conformità alle NdP $_G$:** controllo che il documento rispetti tutte le regole procedurali e strutturali definite nelle presenti norme;
- **completezza delle informazioni:** verifica che non manchino sezioni obbligatorie, che le tabelle siano complete e che i diagrammi siano accompagnati da didascalie esplicative.

Durante questa fase:

- il documento viene compilato localmente dal verificatore oppure automaticamente dalla pipeline CI, per accertarsi che la generazione del PDF non produca errori;
- il PDF generato viene controllato nella sua resa finale, prestando attenzione a impaginazione, overflow di testo e qualità delle immagini;
- eventuali errori vengono segnalati tramite commenti inline nella Pull Request $_G$, ciascuno riferito alla riga di codice $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}_G$ interessata, in modo che il redattore possa correggerli con precisione.

3.2.2.5 Stato di approvazione

Una volta completata positivamente la verifica, il documento può essere approvato dal Responsabile_G. L'approvazione rappresenta l'atto formale di validazione che trasforma il documento in un artefatto ufficiale del progetto.

L'approvazione:

- rappresenta l'atto formale di validazione: con la sua approvazione, il Responsabile_G attesta che il documento ha superato tutti i controlli e può essere utilizzato come riferimento ufficiale;
- rende il documento ufficiale: da questo momento, il contenuto del documento è vincolante per le attività successive e può essere esibito al proponente come deliverable di progetto;
- consente il consolidamento nel branch `main`: la Pull Request_G di approvazione viene mergiata, e il documento confluisce nel ramo principale da cui vengono generati i PDF definitivi;
- rende il documento valido come ground truth_G: qualsiasi decisione, requisito o specifica in esso contenuta diventa la verità di riferimento per lo sviluppo e la verifica del prodotto.

3.2.2.6 Stato di manutenzione

Dopo l'approvazione, ogni documento può essere soggetto a manutenzione. La manutenzione è un'attività continua che garantisce l'allineamento tra il documento e lo stato corrente del progetto.

Le modifiche possono derivare:

- da nuove esigenze progettuali, come l'aggiunta di funzionalità che richiedono l'aggiornamento dell'AdR e del PdP;
- da modifiche ai requisiti, che impattano sia i documenti di analisi sia quelli di specifica;
- da correzione di errori, individuati durante le revisioni post-approvazione o durante l'utilizzo del documento come riferimento;
- dall'evoluzione del prodotto, che può comportare la revisione di stime, pianificazioni e diagrammi architetturali.

Ogni modifica determina il ritorno del documento allo stato di redazione, riavviando il ciclo:

1. modifica (redazione su nuovo branch);
2. verifica (Pull Request_G e revisione);
3. approvazione (validazione del Responsabile_G);
4. consolidamento (merge_G in `main` e aggiornamento del PDF).

Questo ciclo garantisce che ogni aggiornamento sia sempre sottoposto agli stessi controlli di qualità previsti per la stesura iniziale.

3.2.3 Attività previste

3.2.3.1 Produzione della documentazione

La produzione comprende tutte le attività di:

- **stesura:** scrittura di nuovi contenuti, seguendo i template e le linee guida;
- **aggiornamento:** modifica di documenti esistenti per adeguarli all'evoluzione del progetto;
- **manutenzione:** interventi correttivi e migliorativi successivi all'approvazione;
- **consolidamento:** integrazione delle versioni approvate nel ramo principale e generazione dei PDF ufficiali.

Il gruppo utilizza $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}_G$ come linguaggio di markup ufficiale per:

- garantire elevata qualità tipografica, professionale e adatta a documenti tecnici;
- separare contenuto e presentazione, permettendo di concentrarsi sulla scrittura senza preoccuparsi dell'impaginazione;
- facilitare il versionamento tramite Git_G , poiché i file `.tex` sono testo puro e quindi facilmente confrontabili;
- automatizzare la generazione di indici, riferimenti incrociati, bibliografia e glossario, riducendo il lavoro manuale e il rischio di errori;
- uniformare la struttura dei documenti, grazie all'uso di classi e comandi personalizzati condivisi tra tutti i documenti del gruppo.

3.2.3.2 Revisione

Ogni modifica deve essere sottoposta a revisione formale prima dell'integrazione nei branch_G principali. La revisione non è un controllo superficiale, ma un'attività strutturata che segue una checklist definita.

La revisione ha lo scopo di:

- prevenire incoerenze tra le diverse parti del documento e tra documenti diversi;
- garantire conformità alle NdP_G , verificando il rispetto di template, nomenclatura e convenzioni;
- identificare errori di contenuto, come dati errati, calcoli sbagliati o affermazioni non supportate;
- validare i contenuti prodotti, assicurandosi che riflettano fedelmente lo stato del progetto e le decisioni prese.

3.2.3.3 Approvazione

L'approvazione finale dei documenti è responsabilità del Responsabile_G. Questi non può delegare tale compito, poiché l'approvazione implica l'assunzione di responsabilità verso il proponente e verso il gruppo.

Il Responsabile_G:

- valuta il risultato della verifica, esaminando i commenti del verificatore_G e accertandosi che tutte le osservazioni siano state risolte;
- approva formalmente il documento, tramite l'accettazione della Pull Request_G di approvazione;
- autorizza il merge_G finale, garantendo che il documento approvato venga effettivamente consolidato;
- richiede eventuali ulteriori modifiche, se ritiene che il documento non abbia ancora raggiunto il livello qualitativo atteso, respingendo la Pull Request_G e motivando la decisione.

3.2.3.4 Archiviazione e pubblicazione

I documenti approvati vengono:

- archiviati nel branch `main`, che contiene esclusivamente versioni stabili e verificate;
- pubblicati tramite GitHub Pages_G, che rende disponibile un sito web con l'elenco dei documenti e i link ai PDF scaricabili;
- conservati come riferimento ufficiale del progetto, in modo che ogni stakeholder possa sempre accedere alla versione più aggiornata.

3.2.3.5 Compiti del Responsabile

Nel processo documentale il Responsabile_G:

- identifica i documenti da produrre, in base alla fase di progetto e agli obblighi del capitolato_G;
- stabilisce le priorità, decidendo quali documenti devono essere completati con urgenza e quali possono essere rinviati;
- definisce le scadenze, coordinandosi con la pianificazione generale e con le date di revisione;
- assegna redattori e verificatori, scegliendo i membri del gruppo in base alle competenze e al carico di lavoro;
- supervisiona l'avanzamento delle attività, monitorando lo stato delle Issue_G e intervenendo in caso di ritardi;

- approva formalmente la documentazione, dopo aver verificato che la revisione sia stata completata con successo;
- redige verbali interni ed esterni, assumendo direttamente il compito di verbalizzazione durante le riunioni.

3.2.3.6 Compiti dell'Amministratore

L'Amministratore_G supporta il processo documentale occupandosi di:

- inserire nell'ITS_G le attività definite dal Responsabile_G, creando Issue_G con descrizione, etichette e collegamenti alla milestone appropriata;
- mantenere aggiornata la documentazione organizzativa, come l'elenco dei membri, i ruoli e le procedure operative;
- gestire la struttura della repository_G, definendo le cartelle, i permessi di accesso e le regole di protezione dei branch;
- monitorare stato e versioni dei documenti, assicurandosi che ogni documento abbia un numero di versione coerente e un registro delle modifiche aggiornato;
- coordinare il configuration management_G documentale, ovvero garantire che tutte le modifiche ai documenti siano tracciate, versionate e integrabili in modo ordinato.

L'Amministratore_G mantiene inoltre aggiornati i seguenti documenti, che costituiscono l'ossatura organizzativa del progetto:

- **Norme di Progetto_G**: aggiornate ogni volta che il gruppo adotta una nuova procedura, uno strumento diverso o modifica le convenzioni esistenti;
- **Piano di Progetto_G**: rivisto a ogni modifica della pianificazione, per riflettere lo stato corrente delle attività e l'allocazione delle risorse;
- **Piano di Qualifica_G**: mantenuto allineato con le metriche effettivamente raccolte e con gli obiettivi di qualità rinegoziati durante il progetto.

3.2.3.7 Redazione dei verbali

La stesura dei verbali è responsabilità del Responsabile_G, che partecipa a tutte le riunioni ed è in grado di coglierne gli elementi salienti.

I verbali vengono redatti a partire:

- dalle discussioni avvenute durante i meeting interni, di cui il Responsabile_G prende appunti in tempo reale;
- dagli incontri con l'azienda proponente, per i quali è particolarmente importante registrare le decisioni e gli impegni presi;
- dalle decisioni emerse durante le attività operative, come l'assegnazione di task o la risoluzione di problemi tecnici.

La struttura standard dei verbali è definita in 3.2.4.2.

3.2.3.8 Redazione dell'Analisi dei Requisiti

La redazione dell'Analisi dei Requisiti_G è affidata esclusivamente all'Analista_G, figura con competenze specifiche nell'elicitazione, classificazione e formalizzazione dei requisiti.

L'Analista_G è responsabile di:

- **identificazione dei requisiti:** attraverso l'analisi del capitolato_G, gli incontri con il proponente e lo studio di sistemi analoghi;
- **classificazione:** ogni requisito viene categorizzato come funzionale, di qualità o di vincolo, e gli viene assegnato un codice univoco;
- **formalizzazione:** i requisiti sono espressi in linguaggio naturale controllato, con l'uso di verbi modali (shall, should, may) e di template predefiniti;
- **validazione dei requisiti del sistema:** l'Analista_G verifica che i requisiti siano coerenti, non ambigui, verificabili e completi, prima di sottoporli all'approvazione del gruppo e del proponente.

3.2.4 Procedure operative

3.2.4.1 Struttura generale dei documenti

Tutti i documenti ufficiali devono rispettare una struttura comune, che garantisce uniformità e facilita la consultazione.

3.2.4.1.1 Frontespizio La prima pagina del documento deve contenere le seguenti informazioni, disposte secondo il template $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}_G$ condiviso:

- logo ufficiale del gruppo, posizionato al centro;
- titolo del documento, in carattere grande e grassetto;
- nome del gruppo, come da registrazione ufficiale;
- identificativo del gruppo;
- elenco dei componenti ordinati in ordine alfabetico, con nome e cognome;
- numero di matricola di ciascun componente, per identificazione univoca;
- versione del documento, nel formato X.Y.Z secondo il semantic versioning_G;
- data di rilascio, nel formato AAAA-MM-GG;
- stato del documento (Stesura, Revisione, Approvato);
- destinatari, ovvero i soggetti a cui il documento è indirizzato;
- indirizzo e-mail ufficiale del gruppo, per eventuali comunicazioni.

3.2.4.1.2 Registro delle modifiche

Il registro delle modifiche:

- viene posizionato a partire dalla seconda pagina, immediatamente dopo il frontespizio;
- segue logica LIFO_G: la modifica più recente compare in cima alla tabella, facilitando la lettura delle ultime variazioni;
- mantiene tracciabilità delle revisioni effettuate, costituendo un riepilogo cronologico dell'evoluzione del documento.

Per ogni modifica devono essere riportati obbligatoriamente:

- **versione**: il numero di versione dopo la modifica;
- **data**: giorno in cui la modifica è stata completata;
- **autore**: nome del redattore che ha apportato la modifica;
- **verificatore_G**: nome di chi ha effettuato la revisione;
- **descrizione delle modifiche**: sintesi chiara ed esaustiva di cosa è stato cambiato, con eventuale riferimento alla Issue_G associata.

3.2.4.1.3 Indice Ogni documento deve contenere un indice gerarchico_G generato automaticamente da L^AT_EX_G, dotato di collegamenti ipertestuali_G alle relative sezioni. L'indice consente una navigazione rapida e viene posto dopo il registro delle modifiche.

3.2.4.2 Struttura dei Verbali

I verbali costituiscono un'eccezione rispetto alla struttura generale definita in 3.2.4.1. Essi non includono il registro delle modifiche. Per loro natura di documento puntuale che fotografa una specifica riunione, l'ultima versione approvata sostituisce integralmente le precedenti; eventuali correzioni vengono incorporate direttamente in una nuova versione del verbale, senza necessità di conservare una cronologia delle revisioni intermedie.

In aggiunta al template standard, i verbali devono presentare una struttura standard, pensata per rendere rapida la consultazione delle informazioni salienti. La struttura è composta dalle seguenti sezioni obbligatorie.

3.2.4.2.1 Informazioni sulla riunione

Devono essere specificati:

- **data**: giorno in cui si è svolta la riunione;
- **orario di inizio**: ora esatta di avvio;
- **orario di conclusione**: ora di termine, per quantificare la durata effettiva;
- **modalità di svolgimento**: indicazione se in presenza, tramite videochiamata (specificando la piattaforma).

3.2.4.2.2 Presenze La sezione deve riportare:

- nome e cognome dei partecipanti, ordinati alfabeticamente per cognome;
- stato di presenza (Presente/Assente);
- eventuali soggetti esterni e relativa azienda, se hanno preso parte alla riunione.

3.2.4.2.3 Ordine del giorno Deve contenere l'elenco puntato degli argomenti previsti, così come comunicato prima della riunione. Durante la riunione possono essere affrontati anche argomenti non previsti, che vanno aggiunti in un paragrafo separato "Varie ed eventuali".

3.2.4.2.4 Discussione e sintesi Contiene:

- resoconto delle discussioni, con una sintesi per ogni punto all'ordine del giorno;
- decisioni prese, evidenziate con un formato che le renda immediatamente individuabili (es. grassetto o rientro);
- brainstorming_G effettuato, con la registrazione delle idee emerse e delle valutazioni preliminari;
- problematiche emerse, con l'indicazione di eventuali soluzioni proposte o della necessità di approfondimenti.

3.2.4.2.5 Attività da svolgere La sezione deve includere, per ogni attività individuata:

- task_G assegnati, con una descrizione sintetica ma non ambigua;
- responsabili, ovvero i membri incaricati di portare a termine il compito;
- scadenze, espresse come data entro cui il task deve essere completato;
- riferimenti alle Issue_G create sull'ITS_G, per consentire il tracciamento automatico dello stato di avanzamento.

3.2.4.3 Versionamento

Per il versionamento della documentazione il gruppo utilizza Git_G, sfruttando appieno le sue capacità di branching e merging.

3.2.4.3.1 Branch Ogni modifica documentale deve essere effettuata su branch_G dedicati derivati da `develop`. Si opera sempre su un branch separato per isolare le modifiche fino al completamento della revisione.

Il workflow_G adottato prevede i seguenti passi sequenziali:

1. **creazione del branch:** il redattore crea un branch con nome descrittivo (es. `docs/pdp-pianificazione`) a partire da `develop`;

2. **modifica del documento:** il redattore apporta le modifiche necessarie, facendo commit frequenti e ben commentati;
3. **commit_G delle modifiche:** ogni commit è atomico e descrive in modo chiaro la modifica effettuata;
4. **push sul repository remoto:** il branch viene caricato su GitHub per consentire la condivisione e l'apertura della Pull Request_G;
5. **apertura della Pull Request_G:** il redattore crea una PR verso `develop`, descrivendo le modifiche e collegando la Issue_G di riferimento;
6. **assegnazione del Verificatore_G:** la PR viene assegnata al verificatore designato, che riceve notifica e può procedere con la revisione.

3.2.4.3.2 Stato dei documenti Ogni documento può assumere uno dei seguenti stati, che devono essere indicati nel frontespizio e sulla dashboard di progetto:

- **Stesura:** il documento è in fase di redazione e non è ancora stato sottoposto a verifica formale;
- **Revisione:** il documento è stato inviato in Pull Request_G ed è in attesa del responso del verificatore_G;
- **Approvato:** il documento ha superato verifica e approvazione ed è stato consolidato nel branch `main`.

Solo i documenti in stato “Approvato” sono considerati `ground truthG` e possono essere utilizzati come riferimento vincolante per le attività di progetto.

3.2.4.3.3 Nomenclatura Le convenzioni di nomenclatura adottate garantiscono l'immediata riconoscibilità del tipo di documento e della sua versione. Esse sono:

- **Verbali Interni:** `VI_AAAA-MM-GG_descrizione-breve.pdf`
dove `AAAA-MM-GG` è la data della riunione e la descrizione breve riassume l'oggetto principale.
- **Verbali Esterni:** `VE_AAAA-MM-GG_descrizione-breve.pdf`
analogo al precedente, ma con prefisso `VE`.
- **Documenti Generali:** `nome-documento.vX.Y.Z.pdf`
dove `nome-documento` è un identificativo univoco (es. `Norme-di-Progetto`) e `X.Y.Z` è il numero di versione.

3.2.4.3.4 Commit e Pull Request Ogni commit_G e Pull Request_G deve rispettare regole precise per garantire tracciabilità e comprensibilità:

- introdurre modifiche significative: non sono ammessi commit vuoti o con modifiche irrilevanti; ogni commit deve rappresentare un progresso tangibile;

- essere associato a una Issue_G : nel messaggio di commit e nella descrizione della PR deve comparire il riferimento alla Issue_G (es. `Closes #42`) per collegare automaticamente la modifica al task;
- possedere descrizioni chiare e comprensibili: il messaggio di commit deve riassumere in modo conciso la modifica, mentre la descrizione della PR può fornire dettagli aggiuntivi, motivazioni e screenshot;
- rispettare le convenzioni stabilite dal gruppo: in particolare, seguire il formato `<tipo>: <descrizione>` per i commit (es. `docs: aggiorna procedura di verifica`).

3.2.4.4 Processo di verifica tramite Pull Request

Il workflow_G di verifica documentale, già descritto nella sezione dedicata alla verifica, si applica integralmente ai documenti e si articola nei seguenti passi:

1. **creazione del branch dedicato**: il redattore crea un branch a partire da `develop`;
2. **modifica del documento**: vengono apportate le modifiche richieste;
3. **push delle modifiche**: il branch viene caricato sul repository remoto;
4. **apertura della Pull Request_G**: su GitHub, il redattore crea una PR verso `develop`, compilando il template di descrizione;
5. **assegnazione del Verificatore_G**: la PR viene assegnata manualmente o automaticamente al verificatore designato;
6. **collegamento dell'Issue_G pertinente**: nella PR si inserisce il riferimento alla Issue_G da chiudere al momento del merge_G.

Il verificatore_G, durante la revisione:

- controlla il contenuto del documento, riga per riga, basandosi sulla checklist di verifica;
- inserisce commenti inline direttamente sul codice LATEX_G modificato, in modo da rendere immediato il contesto della segnalazione;
- approva la PR se tutte le modifiche sono conformi, oppure richiede modifiche, indicando esattamente quali punti devono essere corretti.

In caso di richiesta di modifiche, il redattore le apporta, aggiorna il branch e la PR viene riesaminata fino all'approvazione finale.

3.2.4.5 Processo di approvazione tramite Pull Request

L'approvazione costituisce l'atto formale che consente il rilascio ufficiale della documentazione. Si distingue dalla verifica perché coinvolge il Responsabile_G, che valuta il documento nella sua interezza e ne autorizza la pubblicazione.

La procedura adottata per l'approvazione è:

1. **creazione del branch:** si crea un branch dedicato denominato `chore/approvazione-<nome-documento>` a partire da `develop`;
2. **aggiornamento della versione Major:** all'interno del documento si incrementa la prima cifra della versione (es. da 1.2.3 a 2.0.0) per segnalare un cambiamento sostanziale, e si aggiorna il registro delle modifiche;
3. **apertura della Pull Request_G:** si apre una PR verso `develop` descrivendo che si tratta della richiesta di approvazione;
4. **assegnazione del Responsabile_G:** la PR viene assegnata al Responsabile_G, unico autorizzato a effettuare il merge_G di approvazione;
5. **approvazione finale:** il Responsabile_G esamina il documento (eventualmente consultando il verificatore) e, se lo ritiene adeguato, approva la PR;
6. **merge_G nel branch develop:** la PR viene mergiata, portando il documento approvato nel ramo di sviluppo;
7. **consolidamento nel branch main:** successivamente, si effettua un merge_G da `develop` a `main`, rendendo la nuova versione disponibile come versione ufficiale e attivando la generazione del PDF definitivo.

Questo flusso garantisce che solo documenti completamente verificati e approvati confluiscono nel ramo principale.

3.2.4.6 Acronimi

La tabella 2 riporta gli acronimi utilizzati in modo ricorrente nella documentazione, con il relativo significato esteso. Essi devono essere espansi alla prima occorrenza in ogni documento.

Acronimo	Significato
VI	Verbale Interno
VE	Verbale Esterno
NdP	Norme di Progetto _G
AdR	Analisi dei Requisiti _G
PdP	Piano di Progetto _G
PdQ	Piano di Qualifica _G
RTB	Requirements and Technology Baseline _G
PB	Product Baseline _G

Tabella 2: Acronimi utilizzati nella documentazione.

3.2.5 Strumenti di supporto

3.2.5.1 $\text{\LaTeX}_G$

$\text{\LaTeX}_G$ è il linguaggio di markup adottato per la produzione della documentazione ufficiale. La scelta è motivata dalla sua capacità di produrre documenti di elevata qualità tipografica, paragonabile a quella delle pubblicazioni scientifiche.

L'utilizzo di $\text{\LaTeX}_G$ consente:

- **elevata qualità tipografica:** gestione automatica della sillabazione, della giustificazione e della spaziatura, con un risultato professionale e uniforme;
- **gestione automatica di riferimenti e indice:** etichette e riferimenti incrociati (label/ref) eliminano il rischio di rimandi errati quando si inseriscono o spostano sezioni;
- **integrazione con Git_G :** i file `.tex` sono testo semplice, quindi versionabili, confrontabili e mergeabili con gli stessi strumenti del codice;
- **facilità di manutenzione:** la separazione tra struttura e contenuto, insieme all'uso di comandi personalizzati, permette di modificare lo stile dell'intera documentazione agendo su pochi file di configurazione.

3.2.5.2 GitHub_G

GitHub_G è la piattaforma di hosting scelta per tutti i repository $_G$ del progetto. Oltre al versionamento del codice, essa supporta integralmente il processo documentale grazie alle seguenti funzionalità:

- **versionamento:** ogni modifica ai documenti è tracciata tramite commit $_G$, con possibilità di revert e confronto tra versioni;
- **gestione delle Pull Request $_G$:** la revisione documentale si svolge interamente su PR, con commenti, approvazioni e merge $_G$;
- **gestione delle Issue $_G$:** le attività documentali sono tracciate come Issue $_G$, assegnate ai membri e collegate a milestone;
- **hosting della documentazione:** i PDF generati sono pubblicati tramite GitHub Pages $_G$, rendendoli accessibili a tutti gli stakeholder;
- **tracciamento delle modifiche:** la timeline delle attività e la history dei file consentono di ricostruire l'evoluzione di ogni documento.

Le funzionalità maggiormente utilizzate nel processo documentale sono:

- **GitHub Issues:** per la gestione dei task di redazione, modifica e verifica;
- **GitHub Projects:** per la visualizzazione Kanban dello stato dei documenti (Stesura, Revisione, Approvato);
- **GitHub Pages:** per la pubblicazione automatica dei PDF ufficiali e per l'hosting del sito di documentazione del progetto.

3.2.5.3 Visual Studio Code

Visual Studio Code è l'editor di testo scelto dal gruppo per la redazione dei documenti in $\text{\LaTeX}_G$. Grazie al suo ecosistema di estensioni e all'integrazione nativa con Git_G , supporta efficacemente il workflow documentale adottato.

3.2.5.4 Notion

Notion è un ambiente collaborativo flessibile, impiegato come spazio di lavoro informale per:

- **brainstorming_G**: raccolta di idee, appunti non strutturati e prime bozze di contenuti;
- **stesura preliminare**: scrittura di prime versioni di testi, tabelle e diagrammi in un contesto collaborativo a bassa formalità, da trasferire successivamente in $\text{\LaTeX}_G$;
- **checklist documentali**: elenchi di controllo per la redazione e la verifica, consultabili e spuntabili da tutti i membri.

L'uso di Notion è limitato alla fase preparatoria e di raccolta; tutti i contenuti destinati a diventare ufficiali devono essere convertiti in $\text{\LaTeX}_G$ e inseriti nella repository_G di progetto.

3.3 Verifica

3.3.1 Processo di verifica

La verifica accompagna l'intero ciclo di vita del software, dalla progettazione fino alla manutenzione. L'obiettivo primario è garantire l'efficienza e la correttezza delle attività, istanziando per ogni prodotto un controllo mirato che attesti la conformità ai requisiti specificati.

- Il processo si avvale di tecniche di analisi e di test per assicurare che ogni artefatto soddisfi le attese.
- Per ottenere risultati affidabili, la verifica deve basarsi su criteri oggettivi e procedure ben definite: solo così è possibile stabilire che un prodotto si trovi in uno stato stabile e sia pronto per la successiva fase di validazione.
- La verifica viene applicata a tutti i processi in esecuzione:
 - quando raggiungono un adeguato livello di maturità,
 - oppure a seguito di modifiche del loro stato.

In tali momenti si analizza la qualità dei prodotti e dei processi stessi rispetto agli standard di qualità condivisi.

- Le attività di verifica sono assegnate a verificatori che, seguendo l'ordine dettato dal modello a V, esaminano i prodotti e ne valutano la rispondenza ai requisiti di qualità descritti nel Piano di Qualifica.
- Tutte le attività del processo di verifica devono essere documentate nel Piano di Qualifica, riportando:

- obiettivi,
- risultati attesi,
- risultati ottenuti.

Questo approccio fornisce linee guida per una valutazione accurata, normando il processo e rendendolo ripetibile.

3.3.2 Attività di verifica

3.3.2.1 Verifica dei documenti

Il verificatore svolge un ruolo cruciale nel garantire la qualità e l'accuratezza della documentazione.

- Quando nella Dashboard di documentazione una *issue* compare nella colonna “Da revisionare”, il verificatore deve esaminare il file corrispondente presente nel repository **Sorgente-documenti**.
- Il revisore riceve una notifica via email non appena il redattore completa il proprio lavoro e crea una *Pull Request*.
- All'interno della sezione “Pull requests” di GitHub, il revisore accede alla richiesta di *merge* del branch di redazione verso il branch **main**, può leggere il documento e aggiungere commenti puntuali direttamente sul contenuto.
- I commenti sono visibili ai redattori, favorendo un processo collaborativo di miglioramento.

Per convalidare un documento il verificatore controlla:

- **Correttezza tecnica:** tutte le informazioni devono essere esatte, coerenti e conformi alle norme vigenti.
- **Conformità alle norme:** il documento deve rispettare le linee guida di formattazione, struttura e stile.
- **Consistenza e coerenza:** assenza di discrepanze interne e di contraddizioni con altri documenti correlati.
- **Chiarezza e comprensibilità:** linguaggio adatto al pubblico di destinazione e privo di ambiguità.
- **Revisione grammaticale e ortografica:** il testo deve essere corretto e professionale.

3.3.2.2 Analisi statica

L'analisi statica è condotta sul prodotto senza eseguirlo.

- L'efficacia dipende dall'esperienza e dalla competenza dei verificatori, che impiegano metodi di lettura manuali e automatici.
- Lo scopo è individuare errori formali, violazioni di vincoli, difetti e proprietà indesiderate.
- Le due principali tecniche utilizzate sono il **walkthrough** e l'**inspection**.

3.3.2.3 Walkthrough

Il verificatore esamina l'intero prodotto alla ricerca di difetti, senza focalizzarsi su una tipologia specifica di errore. L'attività promuove la collaborazione tra autore e verificatore e si articola in quattro passaggi:

1. **Pianificazione:** dialogo tra autore e verificatore per identificare proprietà e vincoli di correttezza.
 2. **Lettura:** il verificatore analizza documenti e/o codice, rilevando errori e verificando la conformità ai vincoli stabiliti.
 3. **Discussione:** confronto tra le parti per analizzare gli esiti della lettura e decidere le correzioni necessarie.
 4. **Correzione:** intervento dell'autore per sanare gli errori emersi.
- Il walkthrough è utilizzato principalmente nelle fasi iniziali del progetto, quando i prodotti sono meno complessi e la verifica risulta meno onerosa.
 - Viene applicato alla revisione dei requisiti, alla codifica (per individuare errori logici e problemi di manutenibilità) e alla documentazione tecnica.

3.3.2.4 Inspection

L'inspection mira a rilevare difetti specifici mediante un'analisi mirata, anziché una revisione indiscriminata.

- Prima dell'ispezione vengono definiti gli elementi da controllare, organizzati in apposite *checklist*.
- L'approccio risulta vantaggioso per documenti o codice complessi e strutturati, tipici delle fasi avanzate del progetto, dove un walkthrough consumerebbe troppe risorse.

3.3.2.5 Analisi dinamica e testing

- L'analisi dinamica richiede l'esecuzione effettiva del prodotto e si applica esclusivamente al software, non alla documentazione.
- Consiste nell'esecuzione di casi di test progettati per verificare il comportamento del codice e individuare scostamenti tra risultati ottenuti e attesi.
- Il processo deve essere automatizzato e ripetibile per garantire oggettività di valutazione. Il test rappresenta la tecnica principale di analisi dinamica.

Di seguito sono descritti i diversi livelli di test adottati.

3.3.2.6 Test di unità

- Verificano singole unità di codice (funzioni, metodi) in isolamento, assicurando che ciascuna produca i risultati attesi.
- Sono pianificati durante la progettazione di dettaglio e vanno eseguiti per primi.
- È ammesso l'uso di mock e stub per isolare l'unità dalle dipendenze.
- L'obiettivo è raggiungere la massima copertura dei percorsi interni (*structural coverage*), progettando test che attivino cammini specifici.

3.3.2.7 Test di integrazione

- Verificano la corretta interazione tra più unità quando vengono aggregate in componenti più ampie o nell'intero sistema.
- Sono pianificati durante la progettazione architetturale.
- L'approccio adottato dal team, ove possibile, è **top-down**:
 - si integrano prima le componenti con maggiori dipendenze e valore esterno,
 - ciò consente di disporre tempestivamente delle funzionalità di alto livello,
 - seppur richiedendo l'uso di diversi mock.

3.3.2.8 Test di sistema

- Verificano l'intero sistema software integrato rispetto ai requisiti specificati, simulando scenari d'uso reali e flussi *end-to-end*.
- Assicurano che tutte le componenti collaborino correttamente e che l'applicazione esegua le funzioni previste in modo accurato e affidabile.
- Vengono eseguiti dopo il superamento dei test di integrazione e di regressione, prima dei test di accettazione.

3.3.2.9 Test di regressione

- Garantiscono che le modifiche al codice non introducano nuovi difetti né compromettano funzionalità già esistenti.
- Devono essere eseguiti ogni volta che vengono apportate modifiche, preferibilmente in modo automatico, per assicurare la stabilità continua del sistema.

3.3.2.10 Test di accettazione

- Sono l'ultimo passo prima del rilascio.
- Verificano che il prodotto finale soddisfi le aspettative degli utenti e risponda adeguatamente ai requisiti specificati, tipicamente attraverso l'esecuzione di scenari d'uso concordati con il proponente.

3.3.2.11 Sequenza delle fasi di test

La sequenza operativa delle fasi di test è la seguente:

1. Test di unità;
2. Test di integrazione;
3. Test di regressione (da ripetere a ogni modifica);
4. Test di sistema;
5. Test di accettazione.

3.3.2.12 Codici identificativi dei test

Ogni test è identificato da un codice univoco nel formato `T[tipo][codice]`, dove:

- `[tipo]` può essere:
 - U – test di unità;
 - I – test di integrazione;
 - S – test di sistema;
 - R – test di regressione;
 - A – test di accettazione.
- `[codice]` è un numero progressivo.
 - Se il test è figlio di un altro test, il codice segue il formato `[codice_padre].[codice_figlio]`,
 - dove `codice_padre` identifica univocamente il test genitore all'interno della stessa categoria.

3.3.2.13 Stato dei test

A ogni test viene assegnato uno stato che ne riflette l'esito, registrato nel *Piano di Qualifica* (sezione “Specifica dei test”):

- **NI** (Non Implementato): il test non è ancora stato realizzato.
- **S** (Superato): esito positivo.
- **NS** (Non Superato): esito negativo.

3.3.2.14 Integrazione continua

L'integrazione continua (*Continuous Integration* – CI) è una pratica di sviluppo che automatizza l'integrazione frequente del codice sorgente.

- Invece di attendere il completamento di grandi porzioni di lavoro, gli sviluppatori integrano le modifiche nel ramo principale più volte al giorno.
- Nel progetto, la CI è attivata dalla creazione di una *Pull Request*: quando un programmatore vuole unire le modifiche di un branch dedicato al ramo principale, una pipeline automatica (GitHub Actions) esegue i test definiti, fornendo un feedback immediato sulla qualità dell'integrazione.

3.3.3 Procedure operative

3.3.3.1 Procedura di verifica documentale

1. Il verificatore riceve notifica della *Pull Request* e accede alla pagina della PR nel repository **Sorgente-documenti**.
2. Esamina il documento, aggiunge commenti e, se necessario, richiede modifiche.
3. Una volta constatato il rispetto dei criteri di qualità, il verificatore:
 - accetta la *Pull Request*, risolve eventuali conflitti ed esegue il *merge* nel branch **main**;
 - elimina il branch di redazione;
 - sposta la *issue* corrispondente dalla colonna “Da revisionare” a “Done” nella Dashboard documentazione.
4. Un'automazione GitHub Actions compila il sorgente $\text{\LaTeX}$ e genera il PDF nel branch **main** del repository **Documents**.
 - Per i verbali il PDF riporterà la data di redazione;
 - per tutti gli altri documenti la versione aggiornata.
5. Se la compilazione fallisce, il verificatore riceve una notifica via email e deve consultare la sezione “Actions” di GitHub per diagnosticare e correggere l'errore.

3.3.3.2 Procedura di walkthrough

1. **Pianificazione**: incontro tra autore e verificatore per definire le proprietà e i vincoli di correttezza.
2. **Lettura**: il verificatore analizza il prodotto nella sua interezza, segnando gli errori e le non conformità.
3. **Discussione**: incontro di confronto per analizzare gli esiti della lettura e concordare le correzioni.
4. **Correzione**: l'autore applica le modifiche richieste.

La procedura viene eseguita per ogni prodotto soggetto a walkthrough, tipicamente nelle fasi iniziali del progetto.

3.3.4 Strumenti di supporto

Per automatizzare e rendere ripetibili le attività di verifica dinamica, il team si avvale di strumenti specifici per il frontend e per il backend.

3.3.4.1 Test frontend

- **Vitest**
 - Test runner veloce, nativamente integrato con l’ecosistema Vite.
 - Supporta TypeScript senza configurazione aggiuntiva e offre funzionalità moderne come l’*Hot Module Replacement* durante l’esecuzione dei test.
 - Viene impiegato per eseguire i test di unità e di integrazione del frontend, sfruttando la sua rapidità e la piena compatibilità con il bundler di progetto.
- **React Testing Library**
 - Fornisce utility per testare componenti React concentrandosi sul comportamento visibile all’utente, anziché sui dettagli implementativi.
 - I test interrogano il DOM nello stesso modo in cui lo farebbe un utente (ad esempio cercando elementi per etichetta, ruolo o testo).
 - Viene utilizzato in combinazione con Vitest per scrivere test di unità e integrazione che simulano l’interazione reale con l’interfaccia.
- **Playwright**
 - Framework di test end-to-end che automatizza i browser (Chromium, Firefox, WebKit).
 - Consente di simulare flussi utente completi, interazioni complesse e di verificare l’applicazione in ambienti multi-browser.
 - Viene adottato per i test di sistema e di accettazione del frontend, perché offre un’elevata fedeltà rispetto all’esperienza reale dell’utente finale e si integra con pipeline CI senza richiedere display grafico.

3.3.4.2 Test backend

- **pytest**
 - Potente framework di test per Python, caratterizzato da una sintassi concisa e dalla ricchezza di plugin.
 - Supporta *fixtures* per la gestione dello stato, parametrizzazione dei test e asserzioni estese.
 - Viene scelto per tutti i livelli di test del backend (unità, integrazione e sistema) perché permette di scrivere test chiari, facilmente manutenibili e di integrarli senza sforzo nella pipeline di integrazione continua.
 - La possibilità di marcare i test (es. `@pytest.mark.unit`, `@pytest.mark.integration`) consente di eseguire selettivamente le suite in base al contesto.

3.4 Validazione

3.4.1 Processo di validazione

Secondo lo standard_G ISO/IEC 12207:1995, la validazione è il processo di conferma, tramite dimostrazione oggettiva, che i requisiti specificati per un sistema o un suo componente siano stati soddisfatti. In termini operativi, essa verifica che il prodotto software_G risponda alle esigenze e alle aspettative degli utenti finali, producendo evidenze misurabili quali test superati, dimostrazioni funzionali o collaudi formali.

Il processo di validazione è essenziale per garantire l'allineamento tra ciò che è stato realizzato e gli obiettivi iniziali del progetto. Un aspetto centrale è l'interazione diretta con il proponente_G, finalizzata a:

- raccogliere feedback tempestivi sul comportamento del prodotto;
- verificare che le funzionalità implementate corrispondano ai casi d'uso previsti;
- individuare eventuali scostamenti rispetto alle attese e concordare le azioni correttive.

Affinché il prodotto possa considerarsi validato, è necessario che:

- soddisfi tutti i requisiti obbligatori e desiderabili concordati con il proponente_G, come documentato nell'*Analisi dei Requisiti*;
- sia eseguibile correttamente nell'ambiente di destinazione finale, ovvero in una configurazione che riproduca fedelmente quella di utilizzo reale.

Il superamento del processo di validazione segna il raggiungimento della maturità del prodotto e ne autorizza il rilascio, chiudendo il ciclo di vita del progetto didattico.

3.4.2 Attività di validazione

Le attività di validazione si concentrano sulla valutazione complessiva del prodotto da parte degli stakeholder. In particolare, il **test di accettazione** costituisce il nucleo del processo di validazione e viene progettato per dimostrare che il sistema soddisfa i criteri di accettazione stabiliti.

Le attività comprendono:

- **Esecuzione dei test di accettazione:** tutti i test di tipo A (accettazione) vengono eseguiti nell'ambiente di produzione simulato, verificando che il sistema risponda correttamente agli scenari d'uso rappresentativi concordati con il proponente.
- **Valutazione della copertura dei casi d'uso:** si verifica che ogni caso d'uso individuato nell'*Analisi dei Requisiti* sia stato implementato e superi i relativi test di accettazione.
- **Verifica del soddisfacimento dei requisiti obbligatori:** si controlla che tutti i requisiti classificati come obbligatori siano stati realizzati e superino i test previsti. Requisiti opzionali o desiderabili possono essere valutati separatamente per determinare il livello di completezza raggiunto.

- **Dimostrazione al proponente_G:** viene organizzata una o più sessioni di presentazione del prodotto, durante le quali il team mostra il funzionamento del sistema, illustra i risultati dei test e risponde a eventuali domande.
- **Raccolta e gestione del feedback:** eventuali osservazioni, richieste di modifica o nuovi requisiti emersi durante la dimostrazione vengono registrati come Issue_G e, se approvati, pianificati per un successivo ciclo di sviluppo, verifica e nuova validazione.

3.4.3 Procedure operative

Il processo di validazione è formalizzato attraverso una procedura operativa che ne garantisce la ripetibilità e la tracciabilità.

1. **Preparazione dell'ambiente:** l'ambiente di test viene configurato in modo da riprodurre il più fedelmente possibile l'ambiente di utilizzo finale (stesso sistema operativo, stesse dipendenze software_G, stessa configurazione di rete). I dati utilizzati sono dati di test rappresentativi, privi di informazioni sensibili.
2. **Esecuzione della suite di accettazione:** i test di accettazione (codificati come TA) vengono eseguiti. L'esecuzione può essere automatizzata tramite gli strumenti di testing. I risultati sono registrati automaticamente e riportano lo stato **S** (Superato) o **NS** (Non Superato).
3. **Registrazione degli esiti:** i risultati dei test di accettazione sono documentati nel *Piano di Qualifica*, nella sezione dedicata alla specifica dei test, aggiornando lo stato di ciascun test.
4. **Sessione di dimostrazione:** il Responsabile_G convoca una riunione con il proponente_G, durante la quale:
 - vengono mostrate le funzionalità implementate;
 - vengono illustrati i risultati dei test di accettazione;
 - si discutono eventuali criticità o aspetti da migliorare.
5. **Approvazione formale:** se il proponente_G ritiene il prodotto conforme alle aspettative, viene redatto un verbale esterno che attesta l'esito positivo della validazione. Tale verbale costituisce l'evidenza oggettiva del completamento del processo.
6. **Gestione delle non conformità:** qualora emergano difetti, funzionalità mancanti o nuovi requisiti, essi vengono registrati nell'ITS_G come Issue_G, valutati dal gruppo e, se accettati, portati in un nuovo ciclo di sviluppo, verifica e successiva validazione. La procedura riprende quindi dal punto 1 per la nuova versione del prodotto.
7. **Rilascio:** una volta ottenuta l'approvazione formale e completata la validazione, il prodotto viene considerato pronto per il rilascio. Il branch `main` viene aggiornato con la versione finale e il team procede con le eventuali attività di deployment concordate.

3.4.4 Strumenti di supporto

La validazione si avvale degli stessi strumenti di testing utilizzati per il processo di verifica (§3.2.5), integrati con strumenti di comunicazione e tracciamento.

- **Playwright**: impiegato per l'esecuzione automatizzata dei test end-to-end e di accettazione del frontend, in grado di simulare l'interazione utente su browser reali e di produrre report dettagliati.
- **pytest**: utilizzato per eseguire i test di accettazione lato backend, sfruttando fixture che riproducono l'ambiente di produzione e verificando il soddisfacimento dei requisiti.
- **GitHub Actions**: le pipeline CI configurate eseguono automaticamente l'intera suite di test (inclusi quelli di accettazione) a ogni Pull Request_G e producono log consultabili dal proponente.
- **GitHub Issues**: impiegate per registrare feedback, richieste di modifica e non conformità emerse durante le sessioni di dimostrazione, garantendo tracciabilità e assegnazione delle responsabilità.
- **Google Meet/Discord**: piattaforme utilizzate per condurre le sessioni di dimostrazione a distanza con il proponente_G, con possibilità di condivisione schermo e registrazione dell'incontro.

3.5 Gestione della configurazione

3.5.1 Processo di gestione della configurazione

La gestione della configurazione del progetto è un processo che norma il tracciamento e il controllo delle modifiche apportate a tutti gli artefatti coinvolti nel ciclo di vita del software_G, detti **Configuration Item** (CI). Essa si applica a qualunque categoria di prodotto: documenti, codice sorgente, librerie, script di build, file di configurazione e ogni altro elemento necessario alla realizzazione e manutenzione del sistema.

Secondo lo standard_G ISO/IEC 12207:1995, la gestione della configurazione del software_G è un processo di identificazione, organizzazione e controllo delle modifiche apportate ai prodotti software_G durante il loro intero ciclo di vita.

Gli obiettivi principali del processo sono:

- **identificare** univocamente ogni Configuration Item e le sue versioni;
- **controllare** le modifiche, assicurando che ogni cambiamento sia autorizzato, documentato e verificato;
- **registrare** lo stato di ciascun CI e la storia delle sue modifiche, garantendo la piena tracciabilità;
- **verificare** la completezza e la coerenza della configurazione in ogni istante del progetto.

L'adozione di un processo strutturato di gestione della configurazione consente al gruppo di lavorare in modo parallelo e coordinato, minimizzando i conflitti, facilitando il ripristino di versioni stabili e fornendo una base solida per le attività di verifica e validazione.

3.5.2 Attività di gestione della configurazione

3.5.2.1 Identificazione dei Configuration Item

Tutti gli artefatti prodotti durante il progetto sono trattati come Configuration Item. Essi includono:

- documenti ufficiali (*Norme di Progetto*, *Analisi dei Requisiti*, *Piano di Progetto*, *Piano di Qualifica*, verbali interni ed esterni);
- codice sorgente del prodotto software_G;
- script di build, configurazioni di ambiente e pipeline CI/CD;
- diagrammi UML, risorse grafiche e ogni altro elemento necessario alla realizzazione del sistema.

Ogni CI è univocamente identificato dal proprio percorso all'interno del repository_G e dalla storia delle sue versioni, registrata tramite Git_G. L'identificazione è fondamentale per poter applicare le procedure di controllo e tracciamento.

3.5.2.2 Versionamento

La convenzione per identificare la versione di un documento è il formato **X.Y.Z**, ispirato al *semantic versioning*_G. Le tre componenti hanno il seguente significato:

- **X (Major)**: viene incrementato al raggiungimento di milestone fondamentali, quali la Requirements and Technology Baseline_G (RTB), la Product Baseline_G (PB) ed eventualmente la Customer Acceptance (CA). Un cambiamento del valore X indica una revisione sostanziale del documento, che ne modifica lo stato di approvazione ufficiale.
- **Y (Minor)**: viene incrementato quando vengono apportate modifiche significative al documento, come cambiamenti strutturali, l'aggiunta di nuove sezioni importanti o modifiche sostanziali nel contenuto. L'incremento di Y segnala un'evoluzione rilevante ma non tale da richiedere una nuova baseline_G formale.
- **Z (Patch)**: viene incrementato per modifiche minori o aggiornamenti circoscritti, quali correzioni di errori, miglioramenti marginali, aggiunta di contenuti meno rilevanti o adeguamenti terminologici.

L'incremento di una componente più significativa azzera automaticamente le componenti meno significative. Questa regola mantiene la chiarezza semantica della numerazione ed evita ambiguità.

Ogni variazione di versione deve essere obbligatoriamente registrata nel **registro delle modifiche** del documento, riportando la nuova versione, la data, l'autore, il verificatore_G e una descrizione sintetica delle modifiche apportate. Per il codice sorgente, invece, il versionamento è gestito automaticamente da Git_G tramite i commit_G e non segue la convenzione X.Y.Z; il rilascio di versioni stabili è accompagnato da **tag** annotati con il numero di versione.

3.5.2.3 Gestione dei repository

Il gruppo utilizza una struttura di repository_G organizzata per separare la documentazione dal codice. I repository_G principali sono:

- thebitless.github.io: contiene tutta la documentazione di progetto, i PDF generati e il sito web ufficiale del gruppo. Questo repository funge da archivio centrale della documentazione approvata ed è accessibile pubblicamente.
- **PoC**: contiene il sorgente del *Proof of Concept*_G del progetto, inclusi i moduli software, i test automatici e le configurazioni di build. La separazione consente di gestire con maggiore flessibilità i permessi e i flussi di lavoro specifici per il codice.

Entrambi i repository_G sono gestiti con Git_G e ospitati su GitHub_G, beneficiando delle funzionalità di versionamento, issue tracking e automazione offerte dalla piattaforma.

3.5.3 Procedure operative

3.5.3.1 Workflow di sviluppo: Gitflow

Il team adotta **Gitflow** come workflow_G Git_G per lo sviluppo del prodotto software_G e per la gestione dei documenti. Gitflow definisce un modello di branching rigoroso, basato su branch con ruoli specifici, che consente di sviluppare funzionalità in parallelo, preparare rilasci e intervenire rapidamente su problemi critici, mantenendo sempre stabile il ramo principale.

Il flusso generale di Gitflow prevede i seguenti branch e procedure:

1. **Branch main**: contiene esclusivamente versioni stabili e rilasciate del prodotto. Ogni commit su **main** rappresenta un punto di rilascio ufficiale ed è marcato con un tag annotato che riporta il numero di versione.
2. **Branch develop**: viene creato a partire da **main** e costituisce il punto di integrazione di tutte le nuove funzionalità. È il branch principale per lo sviluppo attivo e deve rimanere in uno stato compilabile e funzionante.
3. **Branch feature**: creati a partire da **develop**, sono utilizzati per lo sviluppo di nuove funzionalità o miglioramenti. Ogni feature è sviluppata in un branch dedicato, con un nome descrittivo (es. **feature/autenticazione-utente**). Al termine dello sviluppo, il branch viene mergiato in **develop** tramite Pull Request_G dopo aver superato la verifica.
4. **Branch release**: creato da **develop** quando si ritiene che lo stato corrente sia prossimo a un rilascio. Questo branch serve per le attività di preparazione al rilascio: correzione di bug minori, aggiornamento della documentazione, ultimi test. Durante questa fase non sono consentite nuove funzionalità, ma solo interventi di stabilizzazione. Al completamento, il branch **release** viene unito sia in **main** (per produrre il rilascio) sia in **develop** (per allineare le correzioni).
5. **Branch hotfix**: creato da **main** in caso di problemi critici rilevati nell'ambiente di produzione. Il branch **hotfix** consente di risolvere rapidamente il bug senza interferire con lo sviluppo in corso su **develop**. Una volta risolto il problema, il branch viene unito sia in **main** (per aggiornare il rilascio) sia in **develop** (per garantire coerenza tra le versioni future).

3.5.3.2 Gestione delle Pull Request

Dopo aver completato le attività_G nel proprio branch_G, il responsabile dello sviluppo è tenuto ad avviare una **Pull Request** per incorporare le modifiche effettuate nel ramo di destinazione (tipicamente **develop** per le feature, **main** per i rilasci).

La procedura per la creazione di una Pull Request è la seguente:

1. Accedere al repository_G GitHub e cliccare sulla scheda “Pull requests”.
2. Cliccare il pulsante “New Pull Request”.
3. Selezionare il branch_G di partenza (quello contenente le modifiche) e il branch_G target (quello di destinazione, ad esempio **develop**).
4. Cliccare il pulsante “Create Pull Request”.
5. Aggiungere un titolo chiaro e descrittivo alla Pull Request, che riassume lo scopo della modifica.
6. Aggiungere una descrizione dettagliata, specificando il contesto, le motivazioni, l’eventuale Issue_G collegata e ogni altra informazione utile per i revisori.
7. Selezionare i verificatori_G che dovranno revisionare la Pull Request.
8. L’assegnatario della Pull Request è, per convenzione, colui che l’ha creata, in qualità di responsabile dell’integrazione.
9. Selezionare le label_G appropriate per categorizzare la Pull Request (es. **documentation**, **bug**, **enhancement**).
10. Selezionare il progetto GitHub Projects di riferimento, per agganciare la PR al pannello Kanban.
11. Selezionare la milestone_G di competenza, se la modifica è associata a una scadenza di progetto.
12. Cliccare il pulsante “Create Pull Request” per confermare l’apertura.
13. Nel caso siano presenti conflitti con il branch target, GitHub segnalerà la situazione e fornirà le istruzioni per risolverli prima di poter procedere con il merge_G. Il responsabile dovrà risolvere manualmente i conflitti, aggiornare il branch e richiedere nuovamente la revisione.

3.5.3.3 Controllo di configurazione e Change Request

Il controllo di configurazione è il processo che disciplina le richieste di modifica (*Change Request*) ai Configuration Item, assicurando che ogni cambiamento sia valutato, autorizzato e tracciato. Seguendo lo standard_G ISO/IEC 12207:1995, il gruppo adotta una procedura strutturata composta dai seguenti passi:

1. Identificazione e registrazione

Ogni richiesta di modifica viene identificata e registrata formalmente attraverso la creazione di una $Issue_G$ nell' ITS_G con label $_G$ **Change request**. La $Issue_G$ deve contenere informazioni dettagliate sulla natura della modifica richiesta, l'urgenza, l'impatto stimato sul sistema $_G$ esistente e, se possibile, una proposta di soluzione. L'identificazione univoca della richiesta tramite $Issue_G$ ne garantisce la tracciabilità per tutta la durata del processo.

2. Valutazione e analisi

La Change Request viene valutata dal team (in particolare dal Responsabile $_G$ e dagli Analisti $_G$) per determinarne la fattibilità tecnica, l'importanza rispetto agli obiettivi di progetto e l'impatto sugli altri CI. Si analizzano i costi e i benefici associati all'implementazione della modifica, considerando l'effort necessario, i rischi introdotti e le eventuali ricadute su pianificazione e budget.

3. Approvazione o rifiuto

Sulla base dell'analisi condotta, il Responsabile $_G$ decide se approvare o respingere la Change Request. La decisione viene motivata e documentata all'interno della $Issue_G$. I criteri di valutazione includono la coerenza con gli obiettivi del capitolato $_G$, la disponibilità di risorse, il rispetto delle scadenze e le priorità espresse dagli stakeholder $_G$.

4. Pianificazione delle modifiche

Se la Change Request viene approvata, essa viene pianificata e integrata nel ciclo di sviluppo. Ciò comporta la sua assegnazione a una milestone $_G$, la stima dell'effort, l'allocazione delle risorse necessarie e l'eventuale aggiornamento del *Piano di Progetto $_G$* per riflettere le nuove attività. Nei casi più impattanti, può rendersi necessaria la rinegoziazione dei tempi di consegna con il proponente $_G$.

5. Implementazione delle modifiche

Le modifiche vengono effettivamente implementate seguendo il normale flusso di sviluppo (branch dedicato, commit $_G$ atomici, test). Durante questa fase, è fondamentale mantenere una traccia accurata di tutto ciò che viene modificato, per consentire una corretta documentazione e, se necessario, la possibilità di un rollback $_G$ a uno stato precedente.

6. Verifica e validazione

Le modifiche apportate vengono sottoposte a verifica per assicurarsi che raggiungano gli obiettivi previsti e non introducano nuovi difetti o regressioni. In caso di modifiche a componenti critiche, si esegue l'intera suite di test di regressione prima di procedere.

7. Documentazione

Tutti i passaggi del processo di gestione della Change Request vengono documentati accuratamente all'interno della $Issue_G$ e, se necessario, nel registro delle modifiche del documento o nel *Piano di Qualifica*. Questa documentazione garantisce la trasparenza del processo decisionale e costituisce una base di conoscenza utile per futuri riferimenti e per l'apprendimento organizzativo.

3.5.4 Strumenti di supporto

Le tecnologie adottate per la gestione dei Configuration Item sono:

3.5.4.1 Git

Le caratteristiche principali di Git sfruttate nel progetto sono:

- **ramificazione e merging efficienti:** la creazione di branch è rapida e leggera, consentendo di adottare workflow complessi come Gitflow senza overhead;
- **tracciamento completo della storia:** ogni modifica è registrata in un commit_G con autore, data e messaggio descrittivo, rendendo possibile ricostruire l'evoluzione di ogni CI;
- **operazioni locali:** la maggior parte delle operazioni (commit , diff , log) avviene in locale, senza necessità di connessione di rete, velocizzando il lavoro quotidiano;
- **supporto per tag e firma digitale:** i rilasci ufficiali vengono marcati con tag annotati e firmati, garantendo l'autenticità delle versioni distribuite.

3.5.4.2 GitHub

GitHub_G è la piattaforma $_G$ web utilizzata per ospitare i repository_G remoti e per fornire servizi integrati di collaborazione. Oltre all'hosting Git_G , le funzionalità impiegate nella gestione della configurazione sono:

- **GitHub Issues:** sistema di ticketing utilizzato per registrare Change Request, bug, task e attività documentali. Ogni Issue_G è identificata da un numero univoco e può essere etichettata, assegnata e collegata a milestone_G e Pull Request.
- **GitHub Projects:** pannello Kanban che offre una vista aggregata dello stato delle Issue_G , permettendo di monitorare l'avanzamento delle attività di configurazione attraverso colonne personalizzate (es. Da fare, In corso, In revisione, Completato).
- **GitHub Actions:** sistema di automazione CI/CD che, oltre a compilare documenti ed eseguire test, può essere configurato per verificare la consistenza della configurazione (ad esempio, controllando che ogni modifica sia associata a una Issue_G e che i branch rispettino le convenzioni).
- **Pull Request:** strumento per la revisione del codice e dei documenti, che impone un controllo obbligatorio prima dell'integrazione, pilastro del controllo di configurazione.
- **protezione dei branch:** regole configurabili che impediscono commit diretti su `main` e `develop`, obbligando l'uso di Pull Request e l'approvazione da parte di revisori designati.

Questi strumenti, combinati tra loro, consentono di implementare un processo di gestione della configurazione disciplinato, trasparente e pienamente integrato con le attività di sviluppo e documentazione.

3.6 Risoluzione dei problemi

3.6.1 Processo di risoluzione dei problemi

Il processo di risoluzione dei problemi ha lo scopo di analizzare e risolvere qualunque problematica (incluse le non conformità) di qualsiasi natura o fonte, rilevata durante lo

sviluppo, la manutenzione o l'esecuzione di altri processi_G. L'obiettivo è fornire un mezzo tempestivo, responsabile e documentato per garantire che tutti i problemi scoperti siano analizzati e risolti e che vengano riconosciute eventuali tendenze.

Questo processo non si limita alla semplice correzione dei difetti, ma si propone di:

- identificare le cause profonde di ogni problema, attraverso tecniche di analisi strutturata;
- adottare misure preventive per evitare il ripetersi delle stesse criticità in futuro;
- promuovere un ambiente di miglioramento continuo, in cui l'apprendimento dagli errori passati contribuisce alla crescita e all'ottimizzazione dei processi_G;
- raccogliere dati quantitativi e qualitativi per alimentare le attività di controllo e le metriche definite nel *Piano di Qualifica_G*.

Un aspetto centrale della risoluzione dei problemi è l'adozione di approcci strutturati e metodologie efficaci, che prevedono:

- la raccolta sistematica di dati sul problema (contesto, passi per riprodurlo, impatto osservato);
- l'analisi delle cause alla radice (*root cause analysis*), per distinguere i sintomi dalle cause reali;
- la valutazione degli impatti su altri componenti, requisiti e attività di progetto;
- la definizione di azioni correttive immediate e di azioni preventive di lungo termine.

È essenziale mantenere una documentazione accurata di tutti i problemi identificati e delle relative soluzioni implementate. Tale documentazione, gestita tramite il sistema di ticketing (Issue_G su GitHub_G), garantisce la trasparenza del processo decisionale, la tracciabilità completa e la possibilità di revisione nel tempo.

3.6.2 Attività di risoluzione dei problemi

3.6.2.1 Gestione dei rischi

La gestione dei rischi è un'attività proattiva che mira a individuare, analizzare e mitigare le potenziali minacce al successo del progetto prima che si trasformino in problemi reali. All'interno del *Piano di Progetto_G*, nella sezione "Analisi dei rischi", il Responsabile_G identifica tutti i rischi di progetto, classificandoli secondo la loro natura e stimando per ciascuno:

- la probabilità di occorrenza;
- il livello di impatto sul progetto (in termini di tempi, costi, qualità);
- le misure di mitigazione da adottare per ridurre la probabilità o contenere l'impatto.

Per ogni fase di avanzamento del progetto, è dedicata una sezione alla documentazione dei problemi effettivamente riscontrati, con un'analisi del loro impatto e una valutazione dell'efficacia delle contromisure adottate. Questo consente di aggiornare periodicamente la mappa dei rischi e di affinare le strategie di mitigazione sulla base dell'esperienza concreta.

3.6.2.2 Identificazione e tracciamento dei problemi

L'identificazione dei problemi è un'attività reattiva, attivata nel momento in cui un membro del gruppo o uno stakeholder_G rileva un comportamento anomalo, un difetto o una non conformità. È obbligatorio notificare immediatamente il problema al gruppo e registrarne formalmente la segnalazione nel sistema di tracciamento delle Issue_G (ITS), in modo che nulla venga perso e ogni segnalazione sia presa in carico.

La segnalazione deve essere il più possibile completa e contenere tutte le informazioni necessarie per consentire la riproduzione e l'analisi del problema. In particolare, deve includere:

- una descrizione chiara e sintetica del problema;
- i passaggi necessari per riprodurre il comportamento anomalo;
- il comportamento atteso in contrasto con quello osservato;
- l'ambiente in cui si è verificato (versione del software_G, sistema operativo, configurazione);
- eventuali allegati (log, screenshot, file di configurazione) utili alla diagnosi.

Ogni problema rilevato durante lo sviluppo o la manutenzione viene registrato come Issue_G con etichetta *bug* e assegnato al membro del gruppo più competente per la sua risoluzione.

3.6.3 Procedure operative

3.6.3.1 Codifica dei rischi

Per garantire univocità e tracciabilità, ogni rischio identificato nel *Piano di Progetto*_G viene codificato secondo la seguente convenzione:

R[Tipologia]-[Codice] : Nome associato al rischio

dove:

- **Tipologia:** indica la natura del rischio. I valori ammessi sono:
 - T – Rischi legati all'utilizzo delle tecnologie (incompatibilità, obsolescenza, difficoltà di apprendimento, mancanza di documentazione);
 - O – Rischi legati all'organizzazione del gruppo (conflitti, indisponibilità di membri, scarsa comunicazione, sovraccarico di lavoro);
 - P – Rischi legati al prodotto (requisiti instabili, complessità architetturale, difettosità latente, performance inadeguate).
- **Codice:** valore numerico incrementale che identifica in modo univoco il rischio all'interno della sua tipologia (es. RT-1, RO-3, RP-2).

Questa codifica consente di riferirsi ai rischi in modo non ambiguo in tutta la documentazione di progetto e di monitorarne l'evoluzione nel tempo.

3.6.3.2 Procedura di segnalazione e gestione dei problemi

Quando un membro del gruppo identifica un problema, deve seguire la procedura operativa descritta di seguito, che garantisce la presa in carico tempestiva e la tracciabilità completa.

1. **Notifica immediata:** il problema viene comunicato al gruppo attraverso il canale di comunicazione dedicato su Discord (canale **problemi** o simile), descrivendo sinteticamente l'anomalia e la componente interessata.
2. **Apertura della Issue_G:** il membro che ha rilevato il problema apre una Issue_G su GitHub, assegnando l'etichetta **bug** e compilando il template di segnalazione con tutte le informazioni richieste (descrizione, passi per riprodurre, comportamento atteso e osservato, ambiente, allegati).
3. **Valutazione e assegnazione:** il Responsabile_G o l'Amministratore_G valuta la Issue_G, ne stima la priorità (alta, media, bassa) e la assegna al membro del gruppo più idoneo per la risoluzione. Se il problema è critico e blocca lo sviluppo, può essere attivata una procedura di hotfix secondo il workflow Gitflow.
4. **Analisi e risoluzione:** l'assegnatario analizza la causa del problema, propone una soluzione e la implementa su un branch dedicato. Durante l'implementazione vengono aggiunti o aggiornati i test automatici per coprire il caso specifico e prevenire regressioni future.
5. **Verifica:** la soluzione viene sottoposta a verifica tramite Pull Request_G, seguendo le procedure descritte in §3.2. Il verificatore_G controlla che il problema sia effettivamente risolto e che i test associati siano superati.
6. **Chiusura:** una volta verificata e mergiata la soluzione, la Issue_G viene chiusa. Nel commento di chiusura si riporta un riepilogo della soluzione adottata e delle eventuali azioni preventive intraprese.
7. **Capitalizzazione:** se il problema è ritenuto significativo, viene discusso durante la successiva riunione di gruppo per condividere le lezioni apprese e valutare l'aggiornamento della lista dei rischi o delle procedure.

Questa procedura assicura che ogni problema sia gestito in modo disciplinato, documentato e verificabile, in linea con i principi di miglioramento continuo.

3.6.4 Strumenti di supporto

Per la gestione del processo di risoluzione dei problemi, il gruppo si avvale dei seguenti strumenti:

- **GitHub Issues:** sistema di ticketing utilizzato per la segnalazione, il tracciamento e la gestione di tutti i problemi rilevati. Ogni problema è rappresentato da una Issue_G con etichetta **bug**, a cui vengono associati label_G, milestone_G e assegnatari. La piattaforma consente di mantenere una cronologia completa delle discussioni, delle soluzioni proposte e delle decisioni prese.

- **GitHub Projects:** la dashboard Kanban consente di monitorare visivamente lo stato delle Issue_G relative ai problemi, spostandole tra le colonne “Da fare”, “In corso”, “In revisione” e “Completato”. Questa vista facilita il coordinamento del gruppo e la prioritizzazione degli interventi.
- **Discord:** il canale di comunicazione dedicato consente di notificare tempestivamente i problemi critici e di discutere in tempo reale le possibili soluzioni, integrandosi con le segnalazioni formali registrate su GitHub.
- **Git:** il sistema di versionamento distribuito permette di isolare la risoluzione di ogni problema su un branch dedicato, di tracciare le modifiche e di associare ogni commit_G alla Issue_G di riferimento, garantendo la piena tracciabilità tra codice e segnalazioni.

4 Processi organizzativi

Lo sviluppo software è un’attività complessa e multidisciplinare, che richiede una pianificazione accurata, un impiego efficiente delle risorse e un controllo rigoroso della qualità. Per governare questa complessità, il gruppo adotta processi organizzativi ben strutturati, in grado di guidare il lavoro, favorire la collaborazione e garantire il conseguimento degli obiettivi.

4.1 Gestione dei Processi

4.1.1 Processo di gestione dei processi

La Gestione dei Processi ha lo scopo di istituire, attuare e migliorare progressivamente i processi che guidano la realizzazione del software, assicurando che essi siano allineati agli obiettivi di progetto e alle attese degli stakeholder_G. Il processo si articola in cinque aree di intervento fondamentali:

1. Definizione dei processi

Consiste nell’individuare e documentare i processi chiave del ciclo di vita software, stabilendo per ciascuno linee guida, procedure e modelli di esecuzione. La formalizzazione rende i processi trasparenti, ripetibili e comunicabili a tutto il gruppo.

2. Pianificazione e monitoraggio

Prevede l’elaborazione di piani dettagliati che descrivono attività, tempistiche, risorse e costi. L’avanzamento viene monitorato con continuità, confrontando lo stato reale con quanto pianificato, in modo da individuare tempestivamente scostamenti e adottare le necessarie azioni correttive.

3. Valutazione e miglioramento continuo

Comporta lo svolgimento di valutazioni periodiche dei processi, al fine di identificare punti di debolezza e aree di possibile ottimizzazione. Sulla base delle evidenze raccolte, vengono definite e implementate azioni correttive e preventive, in un’ottica di miglioramento progressivo.

4. Formazione e competenze

Assicura che ogni membro del gruppo possieda le conoscenze e le abilità necessarie per svolgere efficacemente le attività previste dai processi. La formazione è continua e si avvale sia dello studio individuale sia del confronto e della condivisione interna.

5. Gestione dei rischi di processo

Consiste nell'identificare, analizzare e valutare i rischi che possono influire negativamente sui processi. Per ciascun rischio vengono definite strategie di mitigazione, che vengono monitorate e aggiornate durante l'intero progetto.

4.1.2 Attività di gestione

4.1.2.1 Pianificazione

La pianificazione rappresenta il fondamento della gestione dei processi. Essa è finalizzata a costruire un piano organico e coerente che guidi l'esecuzione di tutte le attività lungo il ciclo di vita del software. Il Responsabile_G coordina la pianificazione, occupandosi di allocare le risorse, stabilire le scadenze e redigere i documenti di piano.

Il piano deve contenere:

- la descrizione dettagliata delle attività da svolgere;
- le risorse necessarie (umane, tecnologiche, infrastrutturali);
- le tempistiche previste per ciascuna attività;
- le tecnologie e gli strumenti impiegati;
- il personale assegnato a ogni compito.

L'obiettivo primario della pianificazione è garantire che ogni membro del gruppo ricopra, nel corso del progetto, tutti i ruoli previsti almeno una volta. Questa rotazione favorisce una distribuzione equilibrata delle responsabilità e un arricchimento trasversale delle competenze.

L'intera pianificazione è formalizzata nel documento *Piano di Progetto_G*, che fornisce una visione completa delle attività e dei compiti necessari al raggiungimento degli obiettivi in ogni periodo del progetto.

4.1.2.2 Assegnazione dei ruoli e responsabilità

Durante il progetto, i membri del gruppo assumono sei ruoli distinti, ciascuno con responsabilità e compiti specifici. I ruoli sono descritti di seguito.

- **Responsabile**

È la figura che coordina il gruppo e funge da punto di riferimento per il proponente_G e per il team. Gestisce le relazioni esterne, pianifica le attività (definendo cosa fare, quando e con quali priorità), valuta i rischi, controlla i progressi, gestisce le risorse umane e approva formalmente la documentazione.

- **Amministratore**

Si occupa del controllo e dell'amministrazione dell'ambiente di lavoro. Gestisce il versionamento della documentazione e la configurazione del prodotto, redige e attua le norme e le procedure per la qualità, amministra le infrastrutture e i servizi a supporto dei processi.

- **Analista**

Possiede competenze avanzate nell'analisi dei requisiti e nella comprensione del dominio applicativo. Il suo compito è identificare, documentare e comprendere a fondo le esigenze del progetto, traducendole in requisiti chiari e dettagliati. Studia i bisogni espliciti e impliciti e redige il documento *Analisi dei Requisiti_G*.

- **Progettista**

È il riferimento per le scelte progettuali e tecnologiche, a partire dal lavoro dell'Analista. Definisce l'architettura del prodotto, prende decisioni tecniche per garantire affidabilità, efficienza e conformità ai requisiti, e redige il *Piano di Qualifica_G*.

- **Programmatore**

È incaricato della scrittura del codice. Implementa le componenti dell'architettura seguendo le specifiche tecniche, realizza codice manutenibile, sviluppa gli strumenti per la verifica e la validazione del codice e redige il *Manuale Utente*.

- **Verificatore**

Ha la responsabilità di ispezionare il lavoro svolto dagli altri membri del team per assicurarne la qualità e la conformità alle attese. Verifica che i prodotti rispettino le *Norme di Progetto* e i requisiti funzionali e di qualità, segnalando tempestivamente eventuali errori o non conformità.

4.1.2.3 Ticketing e tracciamento delle attività

Per gestire in modo trasparente e ordinato le attività, il gruppo utilizza GitHub come sistema di tracciamento delle issue (ITS). L'Amministratore_G crea e assegna le issue in base alle attività individuate dal Responsabile_G, garantendo che ogni compito abbia un assegnatario chiaro e una scadenza definita.

Ogni issue deve contenere:

- un titolo conciso e descrittivo;
- una descrizione testuale dettagliata o, in alternativa, una lista di cose da fare;
- il nome del verificatore, indicato nell'ultima riga della descrizione nel formato "Verificatore: Nome Cognome";
- l'assegnatario (uno o più membri incaricati dello svolgimento);
- una data di scadenza;
- etichette (*label*) per categorizzare l'issue (es. Verbale, Documents, Develop, Bug, Feature);
- etichette aggiuntive per associare l'issue al Configuration Item corrispondente: NdP, PdQ, PdP, AdR, PoC, GlS;
- la milestone_G di riferimento;
- il progetto GitHub Projects a cui l'issue appartiene;
- l'eventuale branch e la Pull Request collegati (se lo sviluppo è in corso).

L'utilizzo delle dashboard di progetto (Dashboard e Roadmap) consente a ogni membro del gruppo di monitorare in tempo reale lo stato di avanzamento delle attività, visualizzando le issue suddivise per fase e per scadenza.

4.1.3 Procedure operative

4.1.3.1 Procedura di pianificazione

La pianificazione segue un flusso strutturato, condotto dal Responsabile_G in collaborazione con l'Amministratore_G:

1. **Identificazione delle attività:** a partire dagli obiettivi di periodo e dagli impegni presi con il proponente, il Responsabile_G individua tutte le attività da svolgere.
2. **Stima dell'effort e delle risorse:** per ogni attività viene stimato il tempo necessario e vengono identificate le competenze richieste.
3. **Assegnazione dei ruoli:** il Responsabile_G assegna i ruoli ai membri del gruppo, garantendo la rotazione e il bilanciamento dei carichi di lavoro.
4. **Redazione del Piano di Progetto:** le attività, le stime e le assegnazioni vengono formalizzate nel *Piano di Progetto*_G, che viene versionato e sottoposto a verifica.
5. **Creazione delle issue:** l'Amministratore_G traduce le attività pianificate in issue su GitHub, assegnandole ai membri designati e impostando le scadenze.
6. **Monitoraggio e aggiornamento:** durante l'esecuzione, il Responsabile_G verifica lo stato di avanzamento e, se necessario, aggiorna il piano, riportando le modifiche nel documento.

4.1.3.2 Procedura di gestione del ciclo di vita delle issue

Il ciclo di vita di una issue, dalla creazione alla chiusura, segue una procedura operativa standard:

1. L'Amministratore_G crea la issue su GitHub, compilandone tutti gli attributi e assegnandola al membro responsabile.
2. L'Amministratore_G sposta la issue nella colonna "To Do" della dashboard di progetto.
3. L'assegnatario crea un branch dedicato su GitHub, seguendo le convenzioni di nomenclatura.
4. Quando l'assegnatario prende in carico il lavoro, sposta la issue dalla colonna "To Do" a "In Progress".
5. Completata l'attività, l'assegnatario apre una Pull Request verso il branch di destinazione.
6. L'assegnatario sposta la issue nella colonna "Da revisionare" della dashboard.
7. Il Verificatore_G designato esamina le modifiche applicando le procedure di verifica.

8. Se la verifica ha esito positivo, il Verificatore_G approva la Pull Request; la issue viene automaticamente chiusa e spostata nella colonna “Done” della dashboard. In caso di esito negativo, il Verificatore_G richiede modifiche e la issue rimane in “Da revisionare” fino alla risoluzione.

4.1.4 Strumenti di supporto

- **GitHub**

Piattaforma centrale per il tracciamento e la gestione delle attività. Le funzionalità impiegate includono:

- **GitHub Issues:** sistema di ticketing per creare, assegnare e monitorare i compiti. Ogni issue è collegabile a milestone, label, branch e Pull Request.
- **GitHub Projects:** dashboard Kanban per visualizzare lo stato delle issue (To Do, In Progress, Da revisionare, Done) e per avere una panoramica immediata dell'avanzamento.
- **Roadmap:** vista temporale che mostra la distribuzione delle issue nel tempo, utile per il monitoraggio delle scadenze.

- **Google Calendar**

Utilizzato per la pianificazione temporale delle attività. Consente di fissare scadenze, organizzare riunioni e sincronizzare gli impegni di tutto il gruppo, inviando promemoria automatici.

- **Discord**

Piattaforma di comunicazione istantanea impiegata per il coordinamento quotidiano. I canali tematici permettono di discutere rapidamente lo stato delle attività, assegnare compiti urgenti e condividere aggiornamenti in tempo reale.

4.2 Coordinamento

4.2.1 Processo di coordinamento

Il coordinamento è il processo che sovrintende alla gestione della comunicazione e all'organizzazione degli incontri tra tutti i soggetti coinvolti nel progetto. Esso abbraccia tanto la comunicazione interna al team quanto quella esterna con il proponente e gli altri stakeholder, assicurando che il flusso informativo sia sempre tempestivo, chiaro e bidirezionale.

Gli obiettivi principali del coordinamento sono:

- garantire l'efficienza operativa, prevenendo ritardi e incomprensioni;
- mantenere tutte le parti costantemente informate sullo stato del progetto e sulle decisioni prese;
- favorire la collaborazione e la coesione del gruppo, creando un ambiente di lavoro positivo e produttivo;
- facilitare lo scambio di idee e la risoluzione collaborativa dei problemi.

Un coordinamento efficace si fonda su regole condivise, ruoli definiti e strumenti adeguati, che insieme permettono di sincronizzare le attività e di prendere decisioni in modo partecipato e documentato.

4.2.2 Attività di coordinamento

Le attività di coordinamento comprendono la gestione delle comunicazioni e l'organizzazione delle riunioni. Esse sono progettate per mantenere allineati tutti i partecipanti al progetto e per consentire un rapido scambio di informazioni.

4.2.2.1 Comunicazioni sincrone

Le comunicazioni sincrone avvengono in tempo reale e permettono un confronto immediato tra i partecipanti. Si distinguono in:

- **Comunicazioni sincrone interne:** sono dedicate al confronto informale tra i membri del team, alla discussione di problemi tecnici e al coordinamento rapido delle attività quotidiane. La loro immediatezza facilita la risoluzione di dubbi e l'allineamento in corso d'opera.
- **Comunicazioni sincrone esterne:** sono utilizzate per gli incontri formali con il proponente, come i SAL (Stato Avanzamento Lavori). Esse richiedono un canale affidabile e accessibile anche a partecipanti esterni al gruppo, e seguono una pianificazione preventiva per garantire la presenza di tutti i soggetti necessari.

4.2.2.2 Comunicazioni asincrone

Le comunicazioni asincrone non richiedono la presenza simultanea dei partecipanti e consentono di gestire le informazioni con maggiore flessibilità oraria. Si suddividono in:

- **Comunicazioni asincrone interne:** servono per scambi rapidi di messaggi, chiarimenti e aggiornamenti informali tra i membri del gruppo. Sono particolarmente utili per comunicazioni logistiche e per mantenere un flusso continuo di informazioni anche al di fuori degli orari di riunione.
- **Comunicazioni asincrone esterne:** rappresentano il canale ufficiale per le comunicazioni con il proponente e per il coordinamento degli incontri. Garantiscono tracciabilità scritta e il livello di formalità adeguato ai rapporti professionali, fornendo una base documentale per eventuali verifiche future.

4.2.2.3 Riunioni interne

Le riunioni interne sono un pilastro del coordinamento. Il gruppo ha fissato un incontro a cadenza settimanale, indicativamente il martedì, che può svolgersi in presenza oppure in modalità telematica. La partecipazione è fortemente raccomandata; qualora la maggioranza dei membri fosse assente, la riunione viene rinviata. Ulteriori riunioni straordinarie possono essere convocate in caso di necessità.

L'importanza di questi momenti risiede nella loro capacità di:

- creare uno spazio dedicato al confronto diretto e alla condivisione di informazioni;
- favorire l'emersione tempestiva di criticità e il loro superamento collettivo;
- rafforzare il senso di appartenenza al gruppo e la responsabilità condivisa verso gli obiettivi;
- garantire che le decisioni siano prese in modo trasparente e con il contributo di tutti.

All'interno delle riunioni interne, il **Responsabile** assume il ruolo di moderatore e coordinatore: guida la discussione, assicura che tutti i punti all'ordine del giorno vengano affrontati e, al termine, redige il verbale interno. La redazione del verbale è affidata al Responsabile non solo per la sua visione d'insieme, ma soprattutto perché il verbale costituisce l'unico documento ufficiale che attesta le decisioni prese e gli impegni assunti. Affidare la stesura a chi ha la massima responsabilità del progetto garantisce accuratezza, imparzialità e coerenza con la pianificazione complessiva.

Subito dopo il Responsabile, l'**Amministratore** svolge un ruolo organizzativo di supporto: prima della riunione collabora alla stesura dell'ordine del giorno e alla convocazione; durante la riunione tiene traccia delle presenze e annota le attività da inserire nell'ITS; al termine, si occupa di creare o aggiornare le issue corrispondenti ai task assegnati e di allineare la dashboard di progetto con le decisioni emerse.

4.2.2.4 Riunioni esterne

Gli incontri con il proponente (o con i committenti) sono organizzati dal Responsabile, che ha il compito di convocarli, definire l'ordine del giorno e coordinare lo svolgimento della riunione. Durante l'incontro, il Responsabile espone i punti di discussione e passa la parola ai membri del gruppo quando sono chiamati a illustrare aspetti specifici.

Tutti i membri del team si impegnano a garantire la propria presenza costante alle riunioni esterne, compatibilmente con eventuali impegni inderogabili. Se un membro non può partecipare, il Responsabile informa tempestivamente il proponente e, se necessario, richiede il rinvio della riunione a una data successiva.

Anche per le riunioni esterne, il verbale viene redatto dal Responsabile. La scelta è motivata dalla necessità di produrre un resoconto ufficiale, preciso e condiviso di quanto discusso e concordato con il proponente. Il verbale esterno costituisce un documento formale che attesta impegni reciproci, scadenze e decisioni, e come tale deve essere redatto da chi ha la responsabilità complessiva del progetto e la piena visibilità su tutti gli aspetti trattati, evitando frammentazioni o interpretazioni difformi.

4.2.3 Procedure operative

4.2.3.1 Procedura per le riunioni interne

1. **Convocazione:** il Responsabile, coadiuvato dall'Amministratore, comunica data, ora e modalità dell'incontro (presenza o telematico) attraverso i canali di comunicazione interni, con un preavviso di almeno 24 ore.

2. **Ordine del giorno:** prima della riunione, il Responsabile e l'Amministratore predispongono un elenco dei punti da discutere, tenendo conto delle segnalazioni ricevute dai membri e delle scadenze imminenti.
3. **Svolgimento:** il Responsabile modera la discussione, assicurando che ogni punto venga affrontato e che tutti i partecipanti abbiano modo di esprimersi. L'Amministratore registra le presenze e prende appunti sulle decisioni e sui task emergenti.
4. **Verbalizzazione:** al termine della riunione, il Responsabile redige il verbale interno secondo la struttura standard definita nelle Norme di Progetto, riportando ordine del giorno, sintesi delle discussioni, decisioni prese e attività da svolgere con relative scadenze e assegnatari.
5. **Follow-up:** entro 24 ore dalla riunione, l'Amministratore crea o aggiorna le issue sull'ITS corrispondenti ai nuovi task e sposta le issue esistenti nelle colonne appropriate della dashboard di progetto. Il verbale viene condiviso in bozza sui canali interni per eventuali commenti, quindi consolidato.

4.2.3.2 Procedura per le riunioni esterne

1. **Convocazione:** il Responsabile contatta il proponente via email, proponendo date e orari compatibili con gli impegni del gruppo. Ottenuta conferma, crea un evento sul calendario condiviso e notifica il team.
2. **Preparazione:** il Responsabile, sentiti i membri del gruppo, definisce l'ordine del giorno e prepara il materiale da presentare (stato avanzamento lavori, dimostrazioni, quesiti).
3. **Svolgimento:** durante la riunione, il Responsabile conduce la discussione. I membri del team intervengono per illustrare le parti di propria competenza. Le decisioni e gli impegni presi vengono annotati dal Responsabile.
4. **Verbalizzazione:** il Responsabile redige il verbale esterno entro 48 ore dall'incontro, utilizzando il template ufficiale. Il verbale viene sottoposto all'approvazione del proponente prima di essere consolidato.
5. **Registrazione e azioni:** i task e gli impegni emersi vengono tradotti in issue dall'Amministratore, collegandoli al verbale e inserendoli nella milestone di riferimento. Il verbale approvato viene archiviato nel repository documentale.

4.2.4 Strumenti di supporto

Gli strumenti adottati per il coordinamento coprono tutte le esigenze di comunicazione, pianificazione e tracciamento.

- **Discord:** piattaforma per le comunicazioni sincrone interne e per i meeting informali. I canali vocali e testuali consentono discussioni in tempo reale e conservano uno storico delle conversazioni, facilitando il reperimento di decisioni passate.
- **Google Meet:** strumento per le riunioni esterne con il proponente, scelto per la facilità di accesso (basta un browser) e per la qualità audio/video, che lo rendono adatto anche a incontri formali con partecipanti esterni.

- **Gmail**: servizio di posta elettronica utilizzato per le comunicazioni asincrone ufficiali verso l'esterno. Viene impiegato per convocare incontri, inviare verbali, scambiare documentazione e mantenere una traccia scritta di ogni comunicazione rilevante.
- **WhatsApp**: applicazione di messaggistica istantanea per le comunicazioni asincrone interne rapide, come aggiornamenti urgenti, chiarimenti veloci e coordinamento logistico. Consente anche conversazioni private tra singoli membri.
- **Google Calendar**: strumento per la pianificazione e la condivisione degli eventi. Il calendario di progetto è accessibile a tutti i membri e include riunioni interne, scadenze e appuntamenti con il proponente, con promemoria automatici per ridurre il rischio di dimenticanze.
- **GitHub**: oltre al tracciamento delle attività, supporta indirettamente il coordinamento grazie alla possibilità di collegare issue, milestone e Pull Request alle decisioni prese nelle riunioni, mantenendo allineati pianificazione e operatività.

4.3 Miglioramento

4.3.1 Processo di miglioramento

Secondo lo standard ISO/IEC 12207:1995, il processo di miglioramento nel ciclo di vita del software ha lo scopo di stabilire, misurare, controllare e migliorare i processi che lo compongono. Si tratta di un processo continuo e trasversale, che non si esaurisce in una singola fase ma accompagna l'intero progetto, garantendo che l'organizzazione impari dall'esperienza e aumenti progressivamente la propria efficacia.

Le finalità del processo di miglioramento sono:

- **valutare** costantemente le prestazioni dei processi adottati;
- **identificare** le aree che presentano inefficienze, colli di bottiglia o risultati insoddisfacenti;
- **pianificare e attuare** azioni correttive e preventive mirate;
- **verificare** l'efficacia delle azioni intraprese e consolidare i miglioramenti ottenuti.

Il processo si basa su un ciclo iterativo di analisi e miglioramento, che trae alimento dalle retrospettive periodiche e dalla documentazione prodotta durante le attività di progetto. In questo modo, il gruppo trasforma le criticità e gli errori in opportunità di crescita, affinando progressivamente metodi, strumenti e dinamiche di lavoro.

4.3.2 Attività di miglioramento

Le attività di miglioramento si articolano in due macrofasi: **analisi** e **implementazione delle azioni correttive**.

4.3.2.1 Analisi

L'analisi è l'attività attraverso la quale il team esamina i processi adottati per valutarne l'efficacia e la correttezza. Essa deve essere condotta con regolarità e a intervalli di tempo costanti, in modo da disporre di dati aggiornati e confrontabili. Un'analisi condotta sporadicamente rischierebbe di non cogliere tempestivamente le criticità, compromettendo la capacità di reazione del gruppo.

L'analisi si concretizza principalmente nelle **retrospettive**: durante ogni riunione interna, il team dedica una parte iniziale del tempo a riflettere sulle attività svolte nell'ultimo periodo. La retrospettiva coinvolge tutti i membri e si focalizza su:

- ciò che ha funzionato bene e va preservato;
- ciò che ha creato ostacoli, rallentamenti o risultati inferiori alle attese;
- le cause profonde dei problemi emersi, distinguendo tra fattori contingenti e criticità strutturali;
- le possibili soluzioni o modifiche da adottare per migliorare.

L'obiettivo è formulare un insieme di azioni correttive concrete, da attuare nel periodo successivo. Questo meccanismo di feedback continuo consente al gruppo di adattarsi rapidamente ai cambiamenti e di elevare progressivamente la qualità del lavoro.

4.3.2.2 Miglioramento

L'attività di miglioramento consiste nell'implementare le azioni correttive definite durante la retrospettiva. Ogni azione viene assegnata a un responsabile e corredata di una scadenza, esattamente come qualsiasi altro task di progetto.

Dopo l'implementazione, l'efficacia dell'azione viene valutata nella retrospettiva successiva: si verifica se il problema è stato risolto, se sono emersi effetti collaterali indesiderati o se l'azione necessita di aggiustamenti. Questo esito viene formalmente documentato, in modo che il gruppo possa conservare memoria delle soluzioni adottate e del loro impatto.

La documentazione delle azioni correttive e del loro esito viene inserita nella sezione "Revisione del periodo precedente" di ogni verbale interno, creando così una traccia storica consultabile in qualsiasi momento.

4.3.3 Procedure operative

4.3.3.1 Procedura di retrospettiva e miglioramento

1. **Preparazione:** prima della riunione, il Responsabile e l'Amministratore raccolgono i dati rilevanti sull'andamento dell'ultimo periodo (stato delle issue, esiti delle verifiche, eventuali criticità segnalate) per fornire al team una base oggettiva su cui discutere.
2. **Retrospettiva:** all'inizio della riunione interna, il Responsabile guida il team in una discussione strutturata, invitando ciascun membro a esprimere osservazioni e proposte. L'Amministratore annota i punti salienti e le azioni correttive proposte.

3. **Formalizzazione delle azioni:** al termine della discussione, il team concorda una lista di azioni correttive. Per ciascuna vengono definiti: descrizione, responsabile dell'implementazione, scadenza. Le azioni vengono registrate come issue nell'ITS con un'etichetta dedicata (ad esempio **improvement**).
4. **Implementazione:** i responsabili attuano le azioni correttive entro la scadenza concordata, aggiornando lo stato delle issue man mano che il lavoro procede.
5. **Verifica e documentazione:** nella riunione successiva, il team esamina lo stato di ogni azione. L'esito (completata con successo, parzialmente efficace, inefficace, non completata) viene discusso e riportato nella sezione "Revisione del periodo precedente" del verbale interno. Le issue relative alle azioni completate vengono chiuse; quelle che necessitano di ulteriori interventi rimangono aperte e vengono riprogrammate.
6. **Capitalizzazione:** le soluzioni che si rivelano particolarmente efficaci vengono valutate per essere integrate stabilmente nelle procedure di progetto. Se opportuno, le Norme di Progetto vengono aggiornate per riflettere il nuovo assetto processuale.

4.3.4 Strumenti di supporto

- **GitHub Issues:** le azioni correttive vengono tracciate come issue con l'etichetta **improvement**, consentendo di monitorarne lo stato, assegnare responsabili e fissare scadenze. L'integrazione con la dashboard di progetto permette di visualizzarle insieme a tutte le altre attività.
- **GitHub Projects:** la dashboard Kanban offre una vista immediata sullo stato delle azioni di miglioramento (To Do, In Progress, Done), facilitando il follow-up durante le riunioni successive.
- **Discord:** durante le riunioni, i canali testuali possono essere utilizzati per raccogliere in tempo reale spunti e osservazioni dai membri, che l'Amministratore può poi sistematizzare nelle issue.

4.4 Formazione

4.4.1 Processo di formazione

Il processo di formazione ha lo scopo di definire e mantenere gli standard di apprendimento all'interno del team, garantendo che ogni membro possieda le conoscenze e le competenze necessarie per utilizzare in modo consapevole ed efficace le tecnologie adottate per la realizzazione del progetto.

La formazione non è vista come un'attività accessoria, bensì come un investimento continuo sulla qualità del lavoro e sulla capacità del gruppo di affrontare sfide tecniche e organizzative. Essa si integra con il processo di miglioramento, poiché una maggiore preparazione individuale e collettiva riduce l'insorgenza di errori, accelera lo sviluppo e facilita la rotazione dei ruoli.

I principi fondamentali del processo di formazione sono:

- **apprendimento continuo:** ogni membro è incoraggiato ad approfondire in autonomia gli strumenti e le tecnologie, sfruttando materiali condivisi e momenti di studio individuale;

- **condivisione della conoscenza:** le competenze acquisite vengono trasmesse tra i membri del team attraverso momenti di affiancamento strutturato, specialmente in corrispondenza della rotazione dei ruoli;
- **supporto esterno:** il proponente mette a disposizione sessioni formative sulle tecnologie specifiche di progetto, fornendo un quadro chiaro e aggiornato degli strumenti da utilizzare.

Un team adeguatamente formato è in grado di mantenere un ritmo di lavoro sostenibile, di adattarsi rapidamente ai cambiamenti e di produrre artefatti di qualità superiore, riducendo il carico sulle attività di verifica e correzione.

4.4.2 Attività di formazione

Le attività di formazione si articolano su due livelli: individuale e di gruppo. Entrambe concorrono a costruire un bagaglio di competenze omogeneo e a garantire che nessun membro rimanga escluso dalla piena operatività.

4.4.2.1 Formazione individuale

Ciascun membro del team si impegna in un percorso di autoformazione finalizzato a colmare le lacune personali e a prepararsi per il ruolo che andrà a ricoprire. Questa attività è particolarmente rilevante in fase di avvio del progetto e in corrispondenza dei cambi di ruolo.

L'autoformazione si basa su:

- studio di documentazione tecnica, guide e tutorial messi a disposizione dal gruppo o indicati dal proponente;
- sperimentazione pratica sugli ambienti di sviluppo e sugli strumenti di progetto;
- confronto con i membri che hanno già ricoperto il ruolo in precedenza.

Un momento cruciale della formazione individuale è rappresentato dal **passaggio di consegne** che avviene durante la rotazione dei ruoli. Ogni membro, prima di lasciare un ruolo, organizza un incontro con il successivo assegnatario per trasferire le conoscenze pratiche accumulate, le procedure consolidate e le criticità note. Simmetricamente, prima di assumere un nuovo ruolo, il membro si confronta con chi lo ha preceduto per apprendere le competenze richieste, ricevendo indicazioni mirate e materiale di supporto.

4.4.2.2 Formazione di gruppo

La formazione di gruppo è costituita da sessioni formative condotte dal proponente. Questi incontri sono specificamente orientati al trasferimento di competenze sulle tecnologie impiegate nel progetto, con particolare riferimento a:

- strumenti e framework di sviluppo adottati;
- architettura del sistema e scelte progettuali;

- procedure operative definite dal proponente per il dominio applicativo.

La partecipazione del team a tali sessioni è obbligatoria, poiché esse costituiscono un momento insostituibile per allineare le conoscenze di tutti i membri e per ottenere risposte dirette a dubbi e quesiti.

Durante le sessioni, i membri del team sono invitati a prendere appunti e a condividere successivamente i punti salienti sul repository documentale, in modo da creare una base di conoscenza riutilizzabile in futuro e accessibile anche a chi non ha potuto partecipare.

4.4.3 Procedure operative

4.4.3.1 Procedura per il passaggio di consegne (rotazione dei ruoli)

1. **Pianificazione dell'incontro:** il Responsabile, in fase di pianificazione delle attività, individua le coppie di membri coinvolte nella rotazione e fissa un incontro di passaggio consegne prima dell'inizio del nuovo periodo.
2. **Preparazione del materiale:** il membro uscente prepara una sintesi delle attività svolte, delle procedure apprese e delle criticità incontrate. Può avvalersi di documentazione già esistente o produrre appunti specifici.
3. **Svolgimento dell'incontro:** l'incontro si svolge preferibilmente in modalità sincrona, utilizzando gli strumenti di comunicazione del gruppo. Il membro uscente illustra al subentrante le informazioni chiave e risponde ai quesiti. Il membro entrante prende nota degli aspetti più rilevanti.
4. **Condivisione e archiviazione:** gli appunti prodotti durante l'incontro vengono caricati in una sezione dedicata del repository documentale o in uno spazio condiviso, in modo che restino disponibili per consultazioni future.
5. **Verifica dell'efficacia:** dopo un periodo di rodaggio, il Responsabile verifica informalmente con il nuovo assegnatario che il passaggio di consegne sia stato sufficiente e, in caso contrario, dispone ulteriori momenti di affiancamento.

4.4.3.2 Procedura per le sessioni formative con la proponente

1. **Convocazione:** il proponente comunica data, ora e modalità della sessione formativa. Il Responsabile del gruppo provvede a informare tempestivamente tutti i membri e a creare un evento sul calendario condiviso.
2. **Preparazione:** i membri del team, se necessario, studiano materiale propedeutico suggerito dal proponente per affrontare la sessione con un livello minimo di conoscenze pregresse.
3. **Partecipazione:** tutti i membri partecipano alla sessione. Durante l'incontro, è possibile porre domande e richiedere chiarimenti. Un membro designato (solitamente l'Amministratore) prende appunti dettagliati.
4. **Rielaborazione e condivisione:** entro 48 ore dalla sessione, gli appunti vengono rielaborati in un formato leggibile e condivisi con il team tramite il repository documentale o altra piattaforma concordata. In questo modo il contenuto formativo diventa patrimonio comune del gruppo.

5. **Follow-up:** eventuali dubbi residui vengono raccolti e, se necessario, sottoposti al proponente via email per un chiarimento successivo.

4.4.4 Strumenti di supporto

- **Google Meet:** piattaforma utilizzata per le sessioni formative sincrone con il proponente, in virtù della facilità di accesso e della possibilità di condividere schermo e materiali.
- **Discord:** i canali tematici interni possono ospitare discussioni di follow-up alle sessioni formative, domande e risposte rapide tra membri, e momenti di formazione informale tra pari.
- **GitHub:** il repository documentale funge da archivio permanente per gli appunti delle sessioni formative e per il materiale di passaggio consegne. La possibilità di versionare i documenti garantisce che la conoscenza accumulata non vada persa e possa essere aggiornata nel tempo.
- **Google Calendar:** utilizzato per fissare gli appuntamenti di passaggio consegne e per ricordare le sessioni formative, assicurando che tutti i membri siano informati per tempo e possano organizzare i propri impegni.
- **WhatsApp:** per comunicazioni rapide e informali relative alla formazione, come la condivisione di link a risorse utili o la richiesta di chiarimenti su argomenti specifici.

5 Metriche e standard per la Qualità

Il gruppo adotta lo standard **ISO/IEC 9126** quale riferimento per la definizione delle caratteristiche che determinano la qualità del prodotto software. Tale standard individua sei macro-caratteristiche principali, ciascuna delle quali è declinata in una serie di attributi misurabili, utili a valutare in modo oggettivo il livello qualitativo raggiunto dal sistema.

5.1 Funzionalità

La funzionalità esprime la capacità del prodotto di soddisfare i bisogni espliciti e impliciti degli stakeholder, fornendo l'insieme di funzioni necessarie per il raggiungimento degli obiettivi prefissati. Gli attributi che concorrono a definirla sono:

- **Appropriatezza:** indica se il prodotto mette a disposizione le funzioni necessarie a svolgere i compiti previsti in fase di analisi.
- **Accuratezza:** misura la capacità del sistema di fornire risultati corretti e privi di errori non desiderati.
- **Interoperabilità:** esprime l'attitudine del prodotto a interagire con altri sistemi, scambiando dati e servizi secondo protocolli e formati definiti.
- **Conformità:** verifica l'aderenza del prodotto a standard, leggi o regolamenti pertinenti al dominio applicativo.

- **Sicurezza:** riguarda la protezione delle informazioni e delle risorse del sistema, impedendo accessi non autorizzati e preservando la riservatezza, l'integrità e la disponibilità dei dati.

5.2 Affidabilità

L'affidabilità rappresenta la capacità del software di mantenere un livello di prestazioni accettabile nel tempo e in condizioni operative specificate. È scomposta nei seguenti attributi:

- **Maturità:** capacità del sistema di evitare malfunzionamenti dovuti a difetti interni, specialmente in situazioni di carico normale.
- **Tolleranza ai guasti:** attitudine a mantenere un comportamento corretto anche in presenza di anomalie o errori di componenti con cui il sistema interagisce.
- **Recuperabilità:** capacità di ripristinare lo stato operativo e i dati dopo un'interruzione, minimizzando la perdita di informazioni.

5.3 Efficienza

L'efficienza concerne il rapporto tra le prestazioni del prodotto e le risorse impiegate per ottenerle. I principali attributi sono:

- **Tempo di risposta:** tempo necessario al sistema per elaborare una richiesta e restituire un risultato all'utente o a un altro sistema.
- **Utilizzo delle risorse:** misura dell'impiego di memoria, processore, banda di rete e altre risorse durante l'esecuzione delle funzioni previste.

5.4 Usabilità

L'usabilità definisce il grado di facilità con cui gli utenti finali possono interagire con il prodotto. Gli attributi che la caratterizzano sono:

- **Comprensibilità:** chiarezza con cui l'interfaccia e le funzionalità sono presentate all'utente, rendendo immediatamente evidente il loro scopo.
- **Apprendibilità:** tempo e sforzo necessari a un utente inesperto per acquisire familiarità con le operazioni principali del sistema.
- **Operatività:** disponibilità di strumenti che agevolano l'esecuzione dei compiti, come scorciatoie, menu contestuali e automazioni.

5.5 Manutenibilità

La manutenibilità esprime la facilità con cui il prodotto può essere modificato, corretto o esteso nel tempo. I suoi attributi sono:

- **Analizzabilità:** facilità con cui è possibile esaminare il codice sorgente e la documentazione per individuare le cause di un malfunzionamento o di un comportamento anomalo.
- **Modificabilità:** misura dello sforzo richiesto per apportare cambiamenti al software, sia a livello di singoli moduli sia a livello architetturale.
- **Stabilità:** capacità del sistema di non introdurre effetti collaterali indesiderati a seguito di modifiche, mantenendo invariato il comportamento delle parti non interessate.

5.6 Portabilità

La portabilità descrive l'attitudine del prodotto a essere trasferito da un ambiente di esecuzione a un altro, sia esso hardware o software. Gli attributi che la compongono sono:

- **Adattabilità:** facilità con cui il software può essere reso operativo su piattaforme diverse da quella originariamente prevista.
- **Installabilità:** semplicità e rapidità con cui il prodotto può essere installato e configurato nell'ambiente di destinazione.
- **Conformità:** aderenza a norme e convenzioni relative alla portabilità, come l'uso di formati standard o l'astrazione dalle specificità del sistema operativo.

5.7 Nomenclatura delle Metriche

Per identificare in modo univoco e sistematico tutte le metriche impiegate, viene definita la seguente convenzione:

$$\mathbf{M[Tipo][Codice]}$$

Dove:

- **M:** prefisso costante che identifica l'elemento come *Metrica*.
- **Tipo:** indica l'ambito di applicazione e assume uno dei seguenti valori:
 - **PC** (Processo): metriche relative alle attività del ciclo di vita e alla gestione dello sviluppo.
 - **PD** (Prodotto): metriche relative alle caratteristiche tecniche e qualitative del software realizzato.
- **Codice:** numero progressivo a due cifre (a partire da 00), gestito indipendentemente per ciascun tipo.

5.8 Suddivisione secondo ISO/IEC 12207

In coerenza con lo standard **ISO/IEC 12207**, le metriche sono organizzate in base alla natura dei processi a cui si riferiscono. Tale suddivisione consente di valutare la qualità in modo trasversale, coprendo tanto le attività direttamente produttive quanto quelle di supporto e di governo.

- **Processi primari:** costituiscono il nucleo centrale dello sviluppo e includono le attività di fornitura, sviluppo e manutenzione del software. Le metriche associate misurano l'efficacia e l'efficienza di queste attività fondamentali.
- **Processi di supporto:** forniscono sostegno ai processi primari durante l'intero ciclo di vita. Rientrano in questa categoria la documentazione, la verifica, la gestione della configurazione e l'assicurazione della qualità. Le metriche di supporto aiutano a mantenere un ambiente di lavoro organizzato e controllato.
- **Processi organizzativi:** definiscono la struttura e le regole di funzionamento del gruppo di progetto. Comprendono la gestione dei processi stessi, il miglioramento continuo e la formazione. Le metriche organizzative consentono di monitorare l'efficacia della governance e la crescita del team.

Per quanto riguarda il prodotto, le metriche sono invece raggruppate secondo le sei caratteristiche di qualità descritte in precedenza (funzionalità, affidabilità, efficienza, usabilità, manutenibilità, portabilità), così da valutare in modo puntuale il livello qualitativo raggiunto dal software in ciascuna area.

5.8.0.1 Riferimenti

Tutte le definizioni, gli acronimi e le abbreviazioni utilizzati in questo documento sono riportati nel **Glossario**, fornito come documento separato, che garantisce una comprensione uniforme dei termini tecnici e dei concetti rilevanti per il progetto.

6 Metriche di Qualità del Processo

6.1 Processi primari

6.1.1 Fornitura

6.1.1.1 Earned Value (EV)

Codice	MPC01
Formula	$EV_k = BAC \times \text{Percentuale di lavoro completato allo sprint } k$ Legenda: • EV_k: Earned Value allo sprint k • BAC: Budget at Completion (Budget totale previsto) • k: Numero dello sprint di riferimento
Descrizione	L'indicatore Earned Value rappresenta il valore del lavoro completato al termine dello sprint k rispetto al budget totale previsto. Viene calcolato al termine di ciascuno sprint per monitorare il progresso reale del progetto rispetto al piano.
Utilità	L'indicatore è utile per monitorare l'andamento del progetto e valutare se il lavoro svolto è in linea con le aspettative.

Tabella 3: Dettagli Metrica MPC01 - Earned Value

6.1.1.2 Planned Value (PV)

Codice	MPC02
Formula	$PV_k = BAC \times \text{Percentuale di lavoro pianificato allo sprint } k$ Legenda: • PV_k: Planned Value allo sprint k • BAC: Budget at Completion • k: Numero dello sprint di riferimento
Descrizione	L'indicatore Planned Value rappresenta il valore del lavoro pianificato che dovrebbe essere raggiunto al termine dello sprint k . Viene calcolato al termine di ciascuno sprint per monitorare l'avanzamento del progetto rispetto alla pianificazione.
Utilità	L'indicatore è utile per monitorare l'andamento del progetto e valutare se la pianificazione è rispettata. Il valore pianificato non può essere negativo e deve essere inferiore al BAC.

Tabella 4: Dettagli Metrica MPC02 - Planned Value

6.1.1.3 Actual Cost (AC)

Codice	MPC03
Formula	$AC_k = \text{Costo effettivo sostenuto nello sprint } k$ Legenda: • AC_k: Actual Cost allo sprint k
Descrizione	L'indicatore Actual Cost rappresenta il costo effettivo sostenuto per completare il lavoro nello sprint k .
Utilità	L'indicatore è utile per monitorare l'andamento del progetto e valutare se i costi sono in linea con le aspettative.

Tabella 5: Dettagli Metrica MPC03 - Actual Cost

6.1.1.4 Cost Performance Index (CPI)

Codice	MPC04
Formula	$CPI_k = \frac{EV_k}{AC_k}$ Legenda: • CPI_k: Cost Performance Index allo sprint k • EV_k: Earned Value allo sprint k • AC_k: Actual Cost allo sprint k
Descrizione	Il Cost Performance Index rappresenta il rapporto tra il valore del lavoro completato e il costo effettivo sostenuto, aggiornato allo sprint k .
Utilità	Un valore maggiore di 1 indica che il progetto sta rispettando il budget, un valore minore di 1 indica che il progetto sta superando il budget.

Tabella 6: Dettagli Metrica MPC04 - Cost Performance Index

6.1.1.5 Schedule Performance Index (SPI)

Codice	MPC05
Formula	$SPI_k = \frac{EV_k}{PV_k}$ Legenda: • SPI_k: Schedule Performance Index allo sprint k • EV_k: Earned Value allo sprint k • PV_k: Planned Value allo sprint k
Descrizione	Lo Schedule Performance Index rappresenta il rapporto tra il valore del lavoro completato e il valore del lavoro pianificato allo sprint k .
Utilità	Un valore maggiore di 1 indica che il progetto sta rispettando la pianificazione, un valore minore di 1 indica che il progetto sta accumulando ritardi.

Tabella 7: Dettagli Metrica MPC05 - Schedule Performance Index

6.1.1.6 Estimate At Completion (EAC)

Codice	MPC06
Formula	$EAC_k = \frac{BAC}{CPI_k}$ Legenda: • EAC_k: Estimate At Completion calcolato allo sprint k • BAC: Budget at Completion • CPI_k: Cost Performance Index allo sprint k
Descrizione	La metrica Estimate At Completion rappresenta una proiezione del costo finale totale del progetto basata sulla performance attuale rilevata allo sprint k .
Utilità	Utilizza il CPI_k come indicatore di efficienza per correggere la stima iniziale (BAC). Se $CPI_k < 1$, EAC_k sarà maggiore del BAC , indicando un probabile sfioramento del budget.

Tabella 8: Dettagli Metrica MPC06 - Estimate At Completion

6.1.1.7 Estimate To Complete (ETC)

Codice	MPC07
Formula	$ETC_k = EAC_k - AC_k$ Legenda: • ETC_k: Estimate To Complete allo sprint k • EAC_k: Estimate At Completion allo sprint k • AC_k: Actual Cost allo sprint k
Descrizione	La metrica Estimate To Complete stima quanto costerà completare il lavoro rimanente del progetto partendo dallo stato attuale (sprint k).
Utilità	Si calcola sottraendo i costi già sostenuti (AC_k) dalla stima del costo finale totale (EAC_k). Utile per la pianificazione del budget residuo necessario.

Tabella 9: Dettagli Metrica MPC07 - Estimate To Complete

6.1.1.8 Time Estimate At Completion (TEAC)

Codice	MPC08
Formula	$TEAC_k = \frac{\text{Durata pianificata}}{SPI_k}$ Legenda: • $TEAC_k$: Time Estimate At Completion calcolato allo sprint k • SPI_k: Schedule Performance Index allo sprint k
Descrizione	La metrica Time Estimate At Completion proietta la durata finale del progetto basandosi sulla performance temporale rilevata allo sprint k .
Utilità	Utilizza lo SPI_k come indicatore di efficienza temporale per correggere la stima iniziale. Se $SPI_k < 1$, $TEAC_k$ sarà maggiore della durata pianificata, indicando un probabile ritardo.

Tabella 10: Dettagli Metrica MPC08 - Time Estimate At Completion

6.1.2 Sviluppo

6.1.2.1 Requisiti Stability Index (RSI)

Codice	MPC09
Formula	$RSI = \frac{R_{iniziali} - (R_{modificati} + R_{cancellati})}{R_{iniziali} + R_{aggiunti}} \times 100$ Legenda: • RSI: Requisiti Stability Index • $R_{iniziali}$: Numero di requisiti definiti all'inizio del progetto • $R_{modificati}$: Numero di requisiti iniziali che hanno subito modifiche • $R_{cancellati}$: Numero di requisiti iniziali che sono stati rimossi • $R_{aggiunti}$: Numero di nuovi requisiti aggiunti durante il progetto
Descrizione	Indice che misura la stabilità dei requisiti del progetto durante il suo ciclo di vita, quantificando la percentuale di requisiti che non hanno subito modifiche, aggiunte o cancellazioni rispetto al totale. Interpretazione dei valori: • $RSI = 100\%$: requisiti stabili, eccellente analisi iniziale; • $80\% \leq RSI < 100\%$: stabilità accettabile, alcune modifiche sono normali; • $50\% \leq RSI < 80\%$: elevata volatilità, problemi nella definizione dei requisiti; • $RSI < 50\%$: instabilità critica.
Utilità	Valuta la qualità dell'analisi iniziale dei requisiti e identifica problemi nella loro definizione, prevenendo derive temporali e di budget.

Tabella 11: Dettagli Metrica MPC09 - Requisiti Stability Index

6.2 Processi di supporto

6.2.1 Documentazione

6.2.1.1 Indice di Gulpease (IG)

Codice	MPC09
Formula	$IG = 89 + \frac{300 \cdot N_{frasi} - 10 \cdot N_{lettere}}{N_{parole}}$ Legenda: • IG: Indice di Gulpease • N_{frasi}: Numero di frasi nel testo • $N_{lettere}$: Numero di lettere nel testo • N_{parole}: Numero totale di parole nel testo
Descrizione	Indice che misura la leggibilità di un testo in lingua italiana. Interpretazione: • $IG \geq 80$: testo molto facile, istruzione elementare; • $60 \leq IG < 80$: testo facile, istruzione media; • $40 \leq IG < 60$: testo abbastanza difficile, istruzione superiore; • $IG < 40$: testo difficile, istruzione universitaria.
Utilità	Assicura che la documentazione sia comprensibile per il target di riferimento, riducendo ambiguità.

Tabella 12: Dettagli Metrica MPC09 - Indice di Gulpease

6.2.1.2 Indice di Frammentazione (IF)

Codice	MPC10
Formula	$IF = \frac{N_{sezioni} + N_{paragrafi}}{N_{linee}} \cdot 100$ Legenda: • IF: Indice di Frammentazione • $N_{sezioni}$: Numero di sezioni e sottosezioni • $N_{paragrafi}$: Numero di paragrafi • N_{linee}: Numero totale di linee di testo
Descrizione	Misura la suddivisione del testo in unità logiche per evitare "muri di testo". • $IF < 5\%$: testo troppo denso; • $5\% \leq IF \leq 20\%$: bilanciamento ottimale; • $IF > 20\%$: testo eccessivamente frammentato.
Utilità	Facilita la consultazione rapida e la memorizzazione dei concetti strutturando il documento.

Tabella 13: Dettagli Metrica MPC10 - Indice di Frammentazione

6.2.2 Verifica

6.2.2.1 Test Success Rate (TSR)

Codice	MPC11
Formula	$TSR_k = \frac{T_{successo}}{T_{totali}} \cdot 100$ Legenda: • TSR_k: Test Success Rate allo sprint k • $T_{successo}$: Numero di test con esito positivo • T_{totali}: Numero totale di test eseguiti
Descrizione	Percentuale di casi di test superati con successo. Interpretazione: • $TSR = 100\%$: qualità eccellente; • $80\% \leq TSR < 100\%$: qualità accettabile; • $TSR < 70\%$: revisione del codice necessaria.
Utilità	Misura la correttezza del software e l'efficacia delle attività di testing.

Tabella 14: Dettagli Metrica MPC11 - Test Success Rate

6.2.3 Gestione della Qualità

6.2.3.1 Percentuale Metriche Soddisfatte (PMS)

Codice	MPC12
Formula	$PMS_k = \frac{M_{soddisfatte}}{M_{totali}} \cdot 100$ Legenda: • PMS_k: Percentuale Metriche Soddisfatte allo sprint k • $M_{soddisfatte}$: Numero di metriche che rientrano nei range ottimali • M_{totali}: Numero totale di metriche definite
Descrizione	Valutazione della conformità globale del progetto agli standard di qualità prefissati. • $PMS \geq 90\%$: eccellente; • $70\% \leq PMS < 80\%$: sufficiente; • $PMS < 70\%$: insoddisfacente.
Utilità	Offre una visione d'insieme della qualità del progetto e dell'aderenza ai requisiti.

Tabella 15: Dettagli Metrica MPC12 - Percentuale Metriche Soddisfatte

6.3 Processi organizzativi

6.3.1 Gestione dei processi

6.3.1.1 Percentuale Rischi Inattesi (PRI)

Codice	MPC13
Formula	$PRI_k = \frac{R_{inattesi,k}}{R_{totali,k}} \cdot 100$ Legenda: • PRI_k: Percentuale Rischi Inattesi allo sprint k • $R_{inattesi,k}$: Rischi verificatisi non presenti nel piano dei rischi • $R_{totali,k}$: Numero totale di rischi manifestati nello sprint
Descrizione	Misura l'accuratezza dell'analisi dei rischi iniziale. • $PRI = 0$: analisi perfetta; • $PRI > 40\%$: analisi dei rischi inadeguata.
Utilità	Identifica la necessità di migliorare l'attività di prevenzione e gestione delle criticità.

Tabella 16: Dettagli Metrica MPC13 - Percentuale Rischi Inattesi

6.3.1.2 Labor Efficiency (LE)

Codice	MPC14
Formula	$LE_k = \frac{H_{pianificate,k}}{H_{effettive,k}} \cdot 100$ Legenda: • LE_k: Labor Efficiency allo sprint k • $H_{pianificate}$: Ore stimate per le attività completate • $H_{effettive}$: Ore realmente impiegate e rendicontate
Descrizione	Rapporto tra stime temporali e impegno reale. • $LE > 100\%$: team più veloce del previsto; • $LE < 100\%$: team più lento o stime troppo ottimistiche.
Utilità	Monitora la precisione delle stime temporali del team, integrando la visione economica dello SPI.

Tabella 17: Dettagli Metrica MPC14 - Labor Efficiency

7 Metriche di Qualità del Prodotto

7.1 Funzionalità

7.1.0.1 Percentuale Requisiti Obbligatori Soddisfatti (PROS)

Codice	MPD01
Formula	$PROS = \frac{RO_{soddisfatti}}{RO_{totali}} \times 100$ Legenda: • $PROS$: Percentuale Requisiti Obbligatori Soddisfatti • $RO_{soddisfatti}$: Numero di requisiti obbligatori implementati correttamente • RO_{totali}: Numero totale di requisiti obbligatori definiti
Descrizione	Misura la completezza del prodotto rispetto alle funzionalità essenziali. Interpretazione: • $PROS = 100\%$: Prodotto completo e rilasciabile; • $70\% \leq PROS < 90\%$: Prodotto parzialmente funzionante; • $PROS < 70\%$: Prodotto incompleto, non rilasciabile.
Utilità	Determina se il software ha raggiunto il livello minimo di funzionalità necessario per il rilascio.

Tabella 18: Dettagli Metrica MPD01 - Requisiti Obbligatori

7.1.0.2 Percentuale Requisiti Desiderabili Soddisfatti (PRDS)

Codice	MPD02
Formula	$PRDS = \frac{RD_{soddisfatti}}{RD_{totali}} \times 100$ Legenda: • $PRDS$: Percentuale Requisiti Desiderabili Soddisfatti • $RD_{soddisfatti}$: Requisiti desiderabili implementati • RD_{totali}: Requisiti desiderabili definiti
Descrizione	Misura il grado di soddisfacimento dei requisiti che migliorano l'esperienza utente ma non sono critici.
Utilità	Valuta il valore aggiunto fornito oltre le funzionalità minime obbligatorie.

Tabella 19: Dettagli Metrica MPD02 - Requisiti Desiderabili

7.2 Affidabilità

7.2.0.1 Line Coverage (LC)

Codice	MPD04
Formula	$LC = \frac{L_{eseguite}}{L_{totali}} \times 100$ Legenda: • LC: Line Coverage • $L_{eseguite}$: Linee di codice eseguite dai test automatici • L_{totali}: Linee di codice totali del sistema
Descrizione	Indica la percentuale di codice testata. Range ottimali: • $LC \geq 80\%$: Copertura eccellente; • $60\% \leq LC < 80\%$: Copertura buona; • $LC < 40\%$: Copertura critica, rischio elevato di bug.
Utilità	Quantifica l'efficacia dei test nell'individuare difetti e garantire la correttezza logica.

Tabella 20: Dettagli Metrica MPD04 - Line Coverage

7.3 Usabilità

7.3.0.1 Profondità Massima di Navigazione (PMN)

Codice	MPD07
Formula	$PMN = \max_{i=1, \dots, k} \{C_i\}$ Legenda: • PMN: Profondità Massima di Navigazione • C_i: Numero di click per completare l'operazione i-esima • k: Numero totale di operazioni analizzate
Descrizione	Misura il massimo numero di click necessari per completare un'attività. Analizza il "caso peggiore". • $PMN \leq 3$: Eccellente; • $5 < PMN \leq 7$: Accettabile ma migliorabile; • $PMN > 7$: Inefficiente.
Utilità	Garantisce che nessuna funzionalità sia troppo "sepolta" nell'interfaccia.

Tabella 21: Dettagli Metrica MPD07 - Profondità Navigazione

7.4 Efficienza

7.4.0.1 Tempo Medio di Risposta (TMR)

Codice	MPD08
Formula	$TMR = \frac{\sum_{i=1}^k T_i}{k}$ Legenda: • TMR: Tempo Medio di Risposta • T_i: Tempo di risposta per la richiesta i-esima • k: Numero di richieste testate
Descrizione	Tempo medio tra l'invio di una richiesta e la ricezione della risposta. • $TMR \leq 1s$: Eccellente; • $3 < TMR \leq 5s$: Accettabile; • $TMR > 5s$: Inaccettabile (lentezza percepita).
Utilità	Garantisce che il sistema soddisfi le aspettative di fluidità dell'utente.

Tabella 22: Dettagli Metrica MPD08 - Tempo Risposta

7.5 Manutenibilità

7.5.0.1 Complessità Ciclomatica (CC)

Codice	MPD09
Formula	$CC = E - N + 2P$ Legenda: • CC: Cyclomatic Complexity • E: Numero di archi del grafo di controllo • N: Numero di nodi del grafo • P: Numero di componenti connesse (solitamente 1)
Descrizione	Misura il numero di percorsi indipendenti nel codice. • $CC \leq 10$: Codice semplice e manutenibile; • $CC > 30$: Complessità critica, necessario refactoring.
Utilità	Previene la creazione di funzioni eccessivamente intricate e difficili da testare.

Tabella 23: Dettagli Metrica MPD09 - Complessità Ciclomatica

7.6 Portabilità

7.6.0.1 Compatibilità Cross-Browser (CCB)

Codice	MPD10
Formula	$CCB = \frac{B_{supportati}}{B_{target}} \times 100$ Legenda: • CCB: Compatibilità Cross-Browser • $B_{supportati}$: Browser su cui il prodotto funziona correttamente • B_{target}: Totale dei browser target definiti
Descrizione	Percentuale di browser su cui il software mantiene piena operatività.
Utilità	Garantisce che il prodotto sia accessibile alla più ampia base di utenti possibile.

Tabella 24: Dettagli Metrica MPD10 - Compatibilità Browser